

Framework/TotalUpdate/STFTotalUpdate.prw

444 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static oNewModel      := Nil           //Objeto para instanciar o model
5 Static aField         := STFFields()  //Preenche o array com os campos do model
6 Static oStructSL1     := Nil
7 Static oModelMaster   := NIL          //model Master
8 Static nLlValMer      := 0
9 Static nLlVlrTot      := 0
10 Static nLlDes         := 0
11 Static nLlDesNF       := 0
12 Static nLlVlrLiq     := 0
13 Static nLlValBru      := 0
14 Static nLlJuros       := 0
15 Static nLlBonif       := 0
16
17 //-----
18 /*{Protheus.doc} STDTotUpd
19 Funcao responsavel em atualizar os valores contidos no model
20
21 @param      Nil
22 @return     Nil
23 @obs
24 @sample
25 */
26 //-----
27 Function STFTotUpd()
28
29 If !( Len( aField ) > 0 )
30     aField      := STFFields()
31 EndIF
32
33 oNewModel := STFestrTot()
34 oNewModel:SetOperation( 3 )
35 oNewModel:SetDescription("TOTAIS")
36 oNewModel:SetPrimaryKey({"L1_NUM"})
37 oNewModel:Activate()
38
39 Return Nil
40
41 //-----
42 /*{Protheus.doc} STDGetTot
43 Funcao responsavel em chamar a criacao do master e grid
44
45 @param      Nil
46 @return oModel - Retorna o model com toda a estrutura criada
47 @obs
48 @sample
49 */
50 //-----
51 Function STFestrTot()
52

```

```

53 Local oModel      := Nil // Model
54 Local oMasterStr   := Nil
55
56 //Monta estrutura de tabela de cabecario
57 oMasterStr := STFMstStr()
58
59 //Instacia Objeto
60 oModel := MPFormModel():New( 'SL1', /*bPreValidacao*/, /*bPosValidacao*/,/*{ |oMdl| xFRT80MVCGR( oMdl ) }*/, /*bCancel*/ )
61
62 oModel:AddFields( "MasterStr", /*cOwner*/, oMasterStr )
63
64 Return oModel
65
66 //-----
67 /*/{Protheus.doc} STFMstStr
68 Criando a estrutura do master
69
70 @param      Nil
71 @return oStruct - Retorna a estrutura
72 @obs
73 @sample
74 */
75 //-----
76 Function STFMstStr()
77
78 Local nI      := 0 // Variavel de loop
79 Local aArea   := Nil
80 Local cX2Unico := "L1_FILIAL+L1_NUM" //X2_UNICO da tabela SL1
81
82 If ValType(oStructSL1) = '0'
83   oStructSL1:Deactivate()
84   oStructSL1:Activate()
85 Else
86   oStructSL1 := FWFormModelStruct():New() // Estrutura
87
88   aArea := GetArea() // Guarda area
89   //Carrega informacoes da Tabela
90   SX2->( DbSetOrder( 1 ) )
91   SX2->( DbSeek( "SL1" ) )
92
93   If ExistFunc('FWX2Unico')
94     cX2Unico := FWX2Unico("SL1")
95   EndIf
96
97   oStructSL1:AddTable(
98     FWX2CHAVE()
99     StrTokArr( cX2Unico, '+' )
100    FWX2Nome("SL1") )
101
102   RestArea(aArea)
103
104   oStructSL1:AddField( "NUM"

```

```

105         "NUM"                , ; // [02] Desc do campo
106         "L1_NUM"             , ; // [03] Id do Field
107         "N"                   , ; // [04] Tipo do campo
108         16                     , ; // [05] Tamanho do campo
109         2                      , ; // [06] Decimal do campo
110         Nil                    , ; // [07] Code-block de validação do campo
111         Nil                    , ; // [08] Code-block de validação When do campo
112         Nil                    , ; // [09] Lista de valores permitido do campo
113         Nil                    , ; // [10] Indica se o campo tem preenchimento obrigatório
114         Nil                    , ; // [11] Code-block de inicializacao do campo
115         Nil                    , ; // [12] Indica se trata-se de um campo chave
116         Nil                    , ; // [13] Indica se o campo pode receber valor em uma operação de update.
117     )                          // [14] Indica se o campo é virtual
118
119     For nI := 1 To Len(aField)
120
121         oStructSL1:AddField(
122             aField[nI][1]      ,; // [01] Titulo do campo
123             aField[nI][1]      ,; // [02] ToolTip do campo
124             aField[nI][2]      ,; // [03] Id do Field
125             aField[nI][3]      ,; // [04] Tipo do campo
126             aField[nI][4]      ,; // [05] Tamanho do campo
127             aField[nI][5]      ,; // [06] Decimal do campo
128             Nil                ,; // [07] Code-block de validação do campo
129             Nil                ,; // [08] Code-block de validação When do campo
130             Nil                ,; // [09] Lista de valores permitido do campo
131             Nil                ,; // [10] Indica se o campo tem preenchimento obrigatório
132             Nil                ,; // [11] Code-block de inicializacao do campo
133             Nil                ,; // [12] Indica se trata-se de um campo chave
134             Nil                ,; // [13] Indica se o campo pode receber valor em uma operação de update.
135             Nil                ) // [14] Indica se o campo é virtual
136
137     Next nI
138 EndIf
139
140 Return oStructSL1
141
142
143 //-----
144 /*{Protheus.doc} STFFields
145 Retorna o array com os campos a serem add no model
146
147 @param      Nil
148 @return aFils - Retorna os campos que estaraõ composto no model
149 @obs
150 @sample
151 */
152 //-----
153 Function STFFields()
154
155 Local nX      := 0
156 Local aFils := { {"TOTAL"          , "L1_VLRTOT"    , Nil, Nil, Nil, "NF_TOTAL"    },; //01

```

```

157 {"DESCONTO" , "L1_DESCONT" , Nil, Nil, Nil, "NF_DESCTOT" },; //02 - Totalizador de descontos no total (Ex. CTRL + B)
158 {"VL. LIQUIDO" , "L1_VLRLIQ" , Nil, Nil, Nil, "" },; //03
159 {"VL. BRUTO" , "L1_VALBRUT" , Nil, Nil, Nil, "" },; //04
160 {"VL. MERCAD" , "L1_VALMERC" , Nil, Nil, Nil, "NF_VALMERC" },; //05
161 {"VL. DESC NF" , "L1_DESCNF" , Nil, Nil, Nil, "" },; //06
162 {"ENTRADA" , "L1_ENTRADA" , Nil, Nil, Nil, "" },; //07
163 {"JUROS" , "L1_JUROS" , Nil, Nil, Nil, "" },; //08
164 {"VL. ICM" , "L1_VALICM" , Nil, Nil, Nil, "NF_VALICM" },; //09
165 {"VL. IPI" , "L1_VALIPI" , Nil, Nil, Nil, "NF_VALIPI" },; //10
166 {"VL. ISS" , "L1_VALISS" , Nil, Nil, Nil, "NF_VALISS" },; //11
167 {"CREDITO" , "L1_CREDITO" , Nil, Nil, Nil, "" },; //12
168 {"FRETE" , "L1_FRETE" , Nil, Nil, Nil, "" },; //13
169 {"SEGURO" , "L1_SEGURO" , Nil, Nil, Nil, "" },; //14
170 {"DESPESA" , "L1_DESPESA" , Nil, Nil, Nil, "" },; //15
171 {"PESO LIQUID" , "L1_PLIQUI" , Nil, Nil, Nil, "" },; //16
172 {"PESO BRUTO" , "L1_PBRUTO" , Nil, Nil, Nil, "" },; //17
173 {"VOLUME" , "L1_VOLUME" , Nil, Nil, Nil, "" },; //18
174 {"TROCO" , "L1_TROCO1" , Nil, Nil, Nil, "" },; //19
175 {"BR ICMS" , "L1_BRICMS" , Nil, Nil, Nil, "NF_VALICM" },; //20
176 {"ABTO PCC" , "L1_ABTOPCC" , Nil, Nil, Nil, "" },; //21
177 {"VL. PIS" , "L1_VALPIS" , Nil, Nil, Nil, "NF_VALPIS" },; //22
178 {"VL COFI" , "L1_VALCOFI" , Nil, Nil, Nil, "NF_VALCOF" },; //23
179 {"VA. CSLL" , "L1_VALCSLL" , Nil, Nil, Nil, "NF_VALCSL" },; //24
180 {"ACRESCIMO" , "L1_ACRSFIN" , "N", 14, 2 , "NF_ACRESCI" },; //25 - É referenciado do L4 apenas para o tamanho do campo por nao existir no L1.
RONDON - referenciei direto pq o campo nao existe no ambiente colombia. Ver atusx, acho que criei por upd
181 {"TOT NFISCAL" , "L1_NOTFISCAL" , "N", 14, 2 , "" },; //26 - Totalizador Não-Fiscal
182 {"VL. JUROS" , "L1_VLRJUR" , "N", 16, 2 , "" },; //27 - Valor dos juros
183 {"TOT DESC. IT" , "L1_DESCIT" , "N", 16, 2 , "NF_DESCONTO" },; //28 - Totalizador de descontos nos itens
184 {"DESC. NF." , "L1_DESCFIN" , "N", 16, 2 , "" },; //29 - Valor do Desconto Financeiro
185 {"BONIF." , "L1_BONIF" , "N", 16, 2 , "" },; //30 - Bonificação
186 {"TOT NFIS RPS." , "L1_NOTFISRPS" , "N", 16, 2 , "" },; //31 - Totalizador Não-Fiscal de Serviços (RPS)
187 {"Arm.Des.Total" , "L1_ARMDESC" , "N", 16, 2 , "" }} //32 - Total de Desconto aplicado na Venda para regra de desconto do varejo.
188
189
190 dbSelectArea("SX3")
191 dbSetOrder(2)
192
193 //Preenche as colunas Tipo, Tamanho e Decimal do array aFils
194 For nX := 1 To Len(aFils)
195   If SX3->(DbSeek(aFils[nX][2]))
196     aFils[nX][3] := SX3->X3_TIPO
197     aFils[nX][4] := SX3->X3_TAMANHO
198     aFils[nX][5] := SX3->X3_DECIMAL
199   EndIf
200 Next
201
202 Return aFils
203
204
205 //-----
206 /*/{Protheus.doc} STFGetTot
207 Retorna o objeto do model com todos os totais definidos

```

```

208
209 @param      Nil
210 @return oTotal  - Objeto de total
211 @obs
212 @sample
213 /*/
214 //-----
215 Function STFGetTot()
216
217 If ValType( oModelMaster ) <> "0" .AND. ValType( oNewModel ) == "0"
218     oModelMaster := oNewModel:GetModel("MasterStr")
219 EndIf
220
221 Return oModelMaster
222
223
224 //-----
225 /*/{Protheus.doc} STFSetTot
226 Add valor as colunas que não foram alimentadas pela rotina da MatxFis
227
228 @param      cField - Campo a ser alimentado com o novo valor
229 @param      nValue - Valor a ser add ao novo campo
230 @return Nil
231 @obs
232 @sample
233 /*/
234 //-----
235 Function STFSetTot( cField , nValue )
236
237 Default cField := ""
238 Default nValue := 0
239
240 ParamType 0 Var      cField  As Character  Default  ""
241 ParamType 1 Var      nValue   As Numeric   Default  0
242
243 If oModelMaster = NIL
244     oModelMaster := oNewModel:GetModel("MasterStr")
245 EndIf
246
247 oModelMaster:SetValue(cField, nValue)
248
249 Return Nil
250
251
252 //-----
253 /*/{Protheus.doc} STFRefTot
254 Atualiza os valores de totais da MatxFis
255
256 @param      Nil
257 @return Nil
258 @obs
259 @sample

```

```

260 /*/
261 //-----
262 Function STFRefTot(nDescTot)
263 Local nX := 0 //Variavel de loop
264
265 Default nDescTot := 0
266
267 If oModelMaster = NIL .OR. ValType( oModelMaster ) <> "0"
268     oModelMaster := oNewModel:GetModel("MasterStr")
269 EndIf
270
271 For nX := 1 To Len(aField)
272     If aField[nX][6] <> ""
273         oModelMaster:SetValue(aField[nX][2], STBTaxRet( Nil, aField[nX][6] ))
274     EndIf
275 Next nX
276
277 /*/
278     Atualiza os totais
279 /*/
280 STBValues(nDescTot)
281
282 Return Nil
283
284
285 //-----
286 /*/{Protheus.doc} STFTotRestart
287 Reinicia o Totalizador
288
289 @param      Nil
290 @return Nil
291 @obs
292 @sample
293 /*/
294 //-----
295 Function STFTotRestart()
296
297 oNewModel:DeActivate()
298 oNewModel:Activate()
299
300 Return Nil
301
302
303 //-----
304 /*/{Protheus.doc} STFSaleTotal
305 Retorna o Total da venda
306 @param  cFieldTot - Campo do total - Opcional
307 @return nTotSale Retorna o total da venda
308 @obs
309 @sample
310 /*/
311 //-----

```

```

312 Function STFSaleTotal( cFieldTot )
313
314 Local nTotSale      := 0           // Retorna total da venda
315 Local oTotal       := STFGetTot()  // Recebe o Objeto totalizador
316
317 Default cFieldTot := "L1_VLRTOT"   // Campo do total
318
319 If !Empty(cFieldTot)
320     nTotSale := oTotal:GetValue( cFieldTot )
321 EndIf
322
323 Return nTotSale
324
325 //-----
326 /*{Protheus.doc} STFBkpTot
327 Guarda os valores antes da seleção da forma de pagamento
328 @param
329 @return nTotSale Retorna o total da venda
330 @obs
331 @sample
332 */
333 //-----
334 Function STFBkpTot()
335
336 Local oTotal      := STFGetTot()    // Recebe o Objeto totalizador
337
338 nL1ValMer   := oTotal:GetValue( "L1_VALMERC" )
339 nL1VlrTot   := oTotal:GetValue( "L1_VLRTOT" )
340 nL1Des      := oTotal:GetValue( "L1_DESCONT" )
341 nL1DesNF    := oTotal:GetValue( "L1_DESCNF" )
342 nL1VlrLiq   := oTotal:GetValue( "L1_VLRLIQ" )
343 nL1ValBru   := oTotal:GetValue( "L1_VALBRUT" )
344 nL1Juros    := oTotal:GetValue( "L1_JUROS" )
345 nL1Bonif    := oTotal:GetValue( "L1_BONIF" )
346
347 Return Nil
348
349 //-----
350 /*{Protheus.doc} STFRestVlr
351 Restaura os valores antes da seleção da forma de pagamento
352 @param
353 @return nTotSale Retorna o total da venda
354 @obs
355 @sample
356 */
357 //-----
358 Function STFRestVlr()
359
360 /*
361     Atualiza totalizador
362 */
363 STFSetTot( "L1_VLRLIQ" , nL1VlrLiq )

```

```

364 STFSetTot( "L1_VALBRUT" , nL1ValBru )
365 STFSetTot( "L1_DESCONT" , nL1Des )
366 STFSetTot( "L1_VLRTOT" , nL1VlrTot ) // Adiciona no VLRTOT por ultimo para nao duplicar
367 STFSetTot( "L1_BONIF" , nL1Bonif )
368
369 /*/
370     Porcentagem de Desconto no Total
371 /*/
372 STFSetTot( "L1_DESCNF" , STBDiscConvert(nL1Des, "V", nL1ValBru )[2] )
373
374 /*/
375     Atualiza Cesta
376 /*/
377 STDSPBasket( "SL1" , "L1_VALMERC" , nL1ValMer )
378 STDSPBasket( "SL1" , "L1_VLRTOT" , nL1VlrTot )
379 STDSPBasket( "SL1" , "L1_DESCONT" , nL1Des )
380 STDSPBasket( "SL1" , "L1_DESCNF" , nL1DesNF )
381 STDSPBasket( "SL1" , "L1_VLR LIQ" , nL1VlrLiq )
382 STDSPBasket( "SL1" , "L1_VALBRUT" , nL1ValBru )
383 STDSPBasket( "SL1" , "L1_JUROS" , nL1Juros )
384 STDSPBasket( "SL1" , "L1_BONIF" , nL1Bonif )
385
386 If ExistFunc("STIAtuRoda")
387     STIAtuRoda()// Função presente no fonte STIPosMain
388 EndIf
389
390 Return Nil
391
392 //-----
393 /*/{Protheus.doc} STFTotRelease
394 Libera os objetos de totais da venda
395 @param
396 @return
397 @obs
398 @sample
399 /*/
400 //-----
401 Function STFTotRelease()
402
403 If ValType(oNewModel) == "0"
404     oNewModel := FreeObj(oNewModel) //Objeto para instanciar o model
405 EndIf
406 aField := {} //Preenche o array com os campos do model
407
408 If ValType(oStructSL1) == "0"
409     oStructSL1 := FreeObj(oStructSL1)
410 EndIf
411 If ValType(oModelMaster) == "0"
412     oModelMaster := FreeObj(oModelMaster) //model Master
413 EndIf
414 nL1ValMer := 0
415 nL1VlrTot := 0

```

```
416 nLlDes      := 0
417 nLlDesNF    := 0
418 nLlVlrLiq   := 0
419 nLlValBru   := 0
420 nLlJuros    := 0
421 nLlBonif    := 0
422
423 Return
424
425 //-----
426 /*{Protheus.doc} STFTotalServ
427 Retorna o Total de Servicos da venda
428 @return nTotServ Retorna o total de servicos da venda
429 @obs
430 @sample
431 */
432 //-----
433 Function STFTotalServ()
434
435 Local nTotServ := 0 // Retorna total de servicos da venda
436 Local nX      := 0
437
438 For nX := 1 To STDPBLength("SL2")
439     If !STDPBIsDeleted("SL2", nX) .And. LjIsTesISS(STDGPBasket("SL2", "L2_PRODUTO", nX), STDGPBasket("SL2", "L2_TES", nX))
440         nTotServ += STBTaxRet(nX, "IT_TOTAL")
441     EndIf
442 Next nX
443
444 Return nTotServ
```

Framework/UpTables/STDFstUp.prw

495 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDUPDATA.CH"
4 #INCLUDE "XMLXFUN.CH"
5
6
7 Static oWS := NIL
8 Static oXML := NIL
9
10
11 //=====
12 //          Situacoes do _SITUA
13 //
14 // " " - Base Errada, Registro Ignorado.
15 // "00" - Venda Efetuada com Sucesso
16 // "01" - Abertura do Cupom Nao Impressa
17 // "02" - Impresso a Abertura do Cupom
18 // "03" - Item Nao Impresso
19 // "04" - Impresso o Item
20 // "05" - Solicitado o Cancelamento do Item
21 // "06" - Item Cancelado
22 // "07" - Solicitado o Cancelamento do Cupom
23 // "08" - Cupom Cancelado
24 // "09" - Encerrado SL1 (Nao gerado SL4)
25 // "10" - Encerrado a Venda
26 //          Pode nao ter sido impresso o cupom
27 // "TX" - Foi Enviado ao Server (Pdv)
28 // "ER" - Erro ao envia ao server (Pdv)
29 // "EP" - Erro de processamento da venda (Pdv)
30 // "RE" - Ja foi feita nova tentativa de subir (Pdv)
31 // "RX" - Foi Recebido Pelo Server (Server)
32 // "OK" - Foi Processado no Server (Serevr)
33 //          Enviar um OK ao Client que foi Processado
34 // "CP" - Recebido pela Central de PDV
35 // "RY" - Cancelamento Nfc-e
36 // "CX" - Cancelamento enviado first
37 // "C0" - Cancelamento a enviar first
38 //=====
39
40 Function STDFrstUp()
41
42 Return NIL
43
44 //-----
45 /*{Protheus.doc} STDLogCons
46 Enviado o Log para o Console
47
48 @param cMessage      Codigo da estacao
49
50 @return Nil
51 @obs
52 @sample

```

```

53 /*/
54 //-----
55 Static Function STDLogCons( cMessage )
56
57 Default cMessage := ""
58
59 ParamType 0 Var      cMessage      As Character      Default      ""
60
61 Conout(cMessage)
62
63 Return Nil
64
65 //-----
66 /*/{Protheus.doc} STDUpFirstSale
67 Envia as Vendas para o First
68
69 @param aSL1          Array da Venda
70 @param aSL2          Array dos Itens
71 @param aSL4          Array dos Pagamentos
72 @param cEstacaoIni   Estação
73 @param cStatusNum    Retorno da Subida
74
75 @return lRet         Execução com sucesso
76 @obs
77 @sample
78 /*/
79 //-----
80
81 Function STDUpFirstSale( aSL1, aSL2, aSL4, cEstacaoIni, ;
82                          cStatusNum)
83 //retirado conteudo da rotina por não existir a função chamada WSINTEGRACAO PDV()
84 //PDV FIRST não está mais ativo
85 Return .T.
86
87
88 //-----
89 /*/{Protheus.doc} STDUpDtFirst
90 Envia a movimentação para o First
91
92 @Param  cAlias      - Alias que sera usado
93 @Param  nOrder      - Codigo de ordenacao
94 @Param  cChave      - Campos de busca
95 @Param  cBusca      - Conteudo de busca
96 @Param  cConfLocal   - String de confirmacao de gravacao do campo Local
97 @Param  cConfServer  - String de confirmacao de gravacao do campo no server
98 @Param  cEstacaoIni  - Estacao
99 @Param  cFunc        - Funcao a ser executado no server
100
101 @return Nil
102 @obs
103 @sample
104 /*/

```

```

105 //-----
106 Function STDUpDtFirst( cAlias      , nOrder      , cChave      , cBusca      , ;
107                      cConfLocal    , cEstacaoIni, cFunc     )
108
109
110 Local aRegnos      := {}           //Registros
111 Local nX           := 0            // Contador
112 Local cNameField   := ""          // Nome do campo situa para o Alias atual
113 Local aData        := {}           // Array com dados a subir para o server
114 Local lContinua    := .F.         // Controle de execucao
115 Local aArea        := GetArea()    // Guarda area atual
116 Local nCont        := 0            //Contador
117 Local lOk          := .T.          //Dados ok
118 Local aCposOri     := {"E5_DATA", "E5_HISTOR", "E5_VALOR", "E5_PREFIXO", ;
119                      "E5_NUMERO", "E5_BANCO", "E5_AGENCIA", "E5_CONTA"}
120 Local aCpoComp      := {}           //Campos comparação
121 Local aCpoEqua     := {"E5_DATA", "E5_MOEDA", "E5_VALOR", "E5_NATUREZ", ;
122                      "E5_HISTOR", "E5_TIPODOC", "E5_PREFIXO", "E5_NUMERO", ;
123                      "E5_DTDIGIT", "E5_NUMMOV"}
124 Local cSeqAnt       := ""          //Sequencia Anterior
125 Local nC           := 0            //Variavel contadora
126 Local nC2          := 0            //Contador
127 Local nC3          := 0            //Contador
128 Local lMovPar       := .F.         //movimentacao Par
129 Local lRet          := .T.         //Retorno da Rotina
130 Local cXML          := ""          //XML envio
131 Local cError        := ""          //Erro Parseamento
132 Local cWarning      := ""          //Alerta Parseamento
133 Local cPai          := ""          //Node pai
134 Local cPaiAnt       := ""          //Node Pai anterior
135 Local cNode         := ""          //Node
136 Local cTipo         := ""          //Tipo do Node
137 Local cValor        := ""          //Valor do XML
138 Local cEntidade     := ""          //Codigo da Entidade
139 Local cChaveSE5     := ""          //Chave SE5
140 Local cSitua        := ""          //Situação
141
142 Default cAlias      := ""
143 Default nOrder      := 0
144 Default cChave      := ""
145 Default cBusca      := ""
146 Default cConfLocal  := ""
147 Default cFunc       := ""
148
149 ParamType 0 Var cAlias As Character Default ""
150 ParamType 1 Var nOrder As Numeric Default 0
151 ParamType 2 Var cChave As Character Default ""
152 ParamType 3 Var cBusca As Character Default ""
153 ParamType 4 Var cConfLocal As Character Default ""
154 ParamType 7 Var cFunc As Character Default ""
155
156 // Formata campo NOMETABELA_SITUA

```

```

157 // Exemplo: Iniciando com "S": SL1->L1_SITUA, caso contrário MDZ->MDZ_SITUA
158
159 If Substr(Upper(cAlias), 1, 1) == "S"
160     cNameField := Substr(cAlias, 2, 2) + "_SITUA"
161 Else
162     cNameField := cAlias + "_SITUA"
163 EndIf
164
165 DbSelectArea(cAlias)
166 DbSetOrder(nOrder)
167
168 If DbSeek(cBusca)
169
170     nCampos := FCount()
171
172     While (cAlias)->(!Eof()) .AND. &(cChave) == cBusca .AND. nCont <= 20
173         aRecnos := {}
174         nCont := nCont + 1
175         nC := 1
176
177         lOk := .T.
178         lMovPar := .T.
179         lContinua := .T.
180         aData := {}
181         aCpoComp := {}
182         If cAlias == "SE5"
183
184             Do While (cAlias)->(!Eof()) .AND. &(cChave) == cBusca .AND. nC <= 2
185
186                 If nC == 1 .AND. E5_RECPEG == "P"
187                     //Primeiro registro é pagamento
188                     cSeqAnt := E5_SEQ
189                     cChaveSE5 := E5_PREFIXO + E5_NUMERO + Dtos(E5_DATA) + E5_NUMMOV+E5_HISTOR
190                     cSitua := FieldGet(FieldPos(cNameField))
191                 ElseIf nC == 1 .AND. E5_RECPEG <> "P"
192                     //Não atualiza os registros
193                     aRecnos := {}
194                     Loop
195                 EndIf
196                 aAdd(aData, {} )
197                 aAdd(aCpoComp, {})
198                 aAdd(aRecnos, Recno())
199
200                 nC3 := 1
201
202                 Do While lMovPar .AND. nC3 <= Len( aCpoEqua)
203                     aAdd(aCpoComp[nC], { aCpoEqua[nC3], FieldGet(FieldPos( aCpoEqua[nC3])), NIL})
204                     If nC > 1
205                         lMovPar := aCpoComp[nC-1][nC3][2] == aCpoComp[nC][nC3][2]
206                     EndIf

```

```

209
210         nC3++
211     EndDo
212
213
214     If nC == 2
215         lMovPar := lMovPar .AND. E5_RECPEG == "R" .AND. Val(cSeqAnt)+1 == Val(E5_SEQ) //Movimentação em pares
216     EndIf
217
218     If lMovPar
219         nC3 := 1
220         Do While nC3 <= len( aCposOri)
221             aAdd(aData[nC], { aCposOri[nC3], FieldGet(FieldPos( aCposOri[nC3])), NIL})
222             nC3++
223         EndDo
224
225     Else //!lMovPar
226         aRechnos := {}
227         dbskip(1)
228         Loop
229     EndIf
230
231     If nC == 1
232         dbskip(1)
233     EndIf
234     nC++
235 EndDo
236
237 If Len(aRechnos) == 2 .AND. Len(aData) <= 2
238
239     nC := 1
240     //Prepar o xml para envio
241
242     oXML := NIL
243     cXML := "<movbancaria></movbancaria>"
244
245     oXML := XMLParser( cXML, "" , @cError, @cWarning)
246
247
248     cPai := "oXML:_movbancaria"
249
250     //pai cabeclaho
251     cTipo := "NOD"
252     cNode := "cabecalho"
253     cValor := nil
254     STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
255
256     //filhos do cabeçalho
257     cTipo := "NOD"
258     cNode := "tipo"
259     cValor := "4"
260     STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)

```

```

261
262         cTipo := "NOD"
263         cNode := "historico"
264         cValor := STDUpArrVal(aData[nC], "E5_HISTOR")
265         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
266
267         cTipo := "NOD"
268         cNode := "valor"
269         cValor := Str(STDUpArrVal(aData[nC], "E5_VALOR"), ,SE5->(TamSx3("E5_VALOR")[2]))
270         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
271
272         cTipo := "NOD"
273         cNode := "documento"
274         cValor := STDUpArrVal(aData[nC], "E5_PREFIX0")+STDUpArrVal(aData[nC], "E5_NUMERO")
275         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
276
277         cTipo := "NOD"
278         cNode := "data"
279         cValor := TRANSFORM(DTOS(STDUpArrVal(aData[nC], "E5_DATA")), "@r 9999-99-99") //YYYY-MM-DD
280         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
281
282 //fim filhos cabeçalho
283 cPai := Left(cPai, Rat(":", cPai)-1) //volta o pai
284
285 Do While (nC <= 2)
286
287 //pai origem/destino
288 cTipo := "NOD"
289 cNode := If(nC == 1, "origem", "destino")
290 cValor := nil
291 STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
292
293 //filhos
294
295         cTipo := "NOD"
296         cNode := "banco"
297         cValor := STDUpArrVal(aData[nC], "E5_BANCO")
298         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
299
300         cTipo := "NOD"
301         cNode := "agencia"
302         cValor := STDUpArrVal(aData[nC], "E5_AGENCIA")
303         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
304
305         cTipo := "NOD"
306         cNode := "conta"
307         cValor := STDUpArrVal(aData[nC], "E5_CONTA")
308         STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
309
310         cTipo := "NOD"
311         cNode := "categoria"
312         cValor := ""

```

```

313             STDUpGTag( @cPai, cNode, cNode, cTipo, cValor)
314
315             cPai := Left(cPai, Rat(":", cPai)-1) //volta o pai
316             //fim filhos origem
317             nC++
318         EndDo
319
320
321         SAVE oXml XMLSTRING cXML
322
323         cXML := Encode64(AllTrim(cXML))
324         //reenvio
325         lContinua := oWS:EnviarDados(cEntidade/*codigo*/, "Movimentacao Bancaria",STDGtEtChv("SE5",cChaveSE5, cEntidade) ,iif(cSitua == "00", "1", "2"),cXML) //1 -
INCLUSA0;2-ALTERACAO;3-EXCLUS
326         If ValType(lContinua) <> "L" .OR. !lContinua
327             STDLogCons(IIf( Empty(GetWscError(3)), GetWscError(1), GetWscError(3) ))
328             lContinua := .F.
329         EndIf
330
331     Endif
332
333
334 EndIf
335
336 If lContinua
337
338     //array de dados
339     For nX := 1 to len(aRecnos)
340         dbGoto(aRecnos[nX])
341         If RecLock( cAlias, .F. )
342             REPLACE &((cAlias)->(cNameField)) WITH cConfLocal
343             &(cAlias)->(dbCommit())
344             &(cAlias)->(MsUnlock())
345
346
347         EndIf
348     Next nX
349 Else
350     STDLogCons(STR0014 ) //"Erro de comunicacao com servidor"
351 EndIf
352
353 /*
354     Por garantia posiciona no registro anterior para depois
355     ir para o primeiro, pois ao alterar um campo do indice
356     os registros serao reordenados.
357 */
358 &(cAlias)->(DbSkip(-1))
359 &(cAlias)->(DbGoTop())
360
361 EndDo
362
363 EndIf

```

```

364 oXML := NIL
365
366 cSeqAnt := NIL
367
368 RestArea(aArea)
369
370 Return Nil
371
372 //-----
373 /*{Protheus.doc} STDUpGTag
374 Cria e alimenta o valor da Tag
375
376 @Param   cPai   - Node Pai
377 @Param   cNode  - Nome Logico do Node
378 @Param   cRealNode - Nome real do Node
379 @Param   cTipo  - Tipo do Node
380 @Param   cValor  - Valor do Node
381 @return  Nil
382 @obs
383 @sample
384 */
385 //-----
386
387 Static Function STDUpGTag( cPai, cNode, cRealNode, cTipo, ;
388                          cValor)
389
390 Default cRealNode := cNode
391
392 XmlNewNode(&cPai, cNode, cNode, cTipo )
393 &(cPai+":" + cNode + ":RealName") := cRealNode
394 If cValor <> NIL
395     &(cPai+":" + cNode + ":Text") := AllTrim(cValor)
396 Else
397     cPai += ":" + cNode
398 EndIf
399
400 Return
401
402 //-----
403 /*{Protheus.doc} STDUpFrDsty
404 Destroi o Objeto WS
405
406 @return  Nil
407 @obs
408 @sample
409 */
410 //-----
411
412 Function STDUpFrDsty()
413
414 If oWS <> NIL
415     oWS := FreeObj(oWS)

```

```

416 EndIf
417
418 Return
419
420 //-----
421 /*{Protheus.doc} STDUpArrVal
422 Retorna o valor dos arrays da Venda
423
424 @Param   aArray      - Array de Dados
425 @Param   cCampo      - Nome do Campo
426 @Param   aHeader     - Header de Campo (SL2/SL4)
427 @Param   lL2L4       - Array SL2/SL4
428 @Param   cValor      - Valor do Node
429 @return  uRet        - Valor
430 @obs
431 @sample
432 */
433 //-----
434
435 Static Function STDUpArrVal(aArray, cCampo, aHeader, lL2L4)
436 Local nPos := 0
437 local uRet := NIL
438
439
440 If lL2L4
441     nPos := aScan(aHeader, { |l| l == AllTrim(cCampo)})
442
443     If nPos > 0
444         uRet := aArray[nPos]
445     EndIf
446 Else
447     nPos := aScan(aArray, { |l| l[1] == AllTrim(cCampo)})
448
449
450     If nPos > 0
451         uRet := aArray[nPos, 02]
452     EndIf
453
454 EndIf
455
456 Return uRet
457
458 //-----
459 /*{Protheus.doc} STDGtEtChv
460 Retorna a Entidade First + Chave
461
462 @Param   cAliasEnt    - Alias da Entidade
463 @Param   cChave       - Chave
464 @Param   cEntidade    - Codigo da Entidade
465 @return  cChaveRet    - Retorno da rotina
466 @obs
467 @sample

```

```
468 /*/
469 //-----
470
471
472 Function STDGtEtChv(cAliasEnt, cChave, cEntidade)
473 Local cChaveRet      := "" //Chave de Retorno
474 Local cFilFirst := SuperGetMv("MV_LJFILIN",.F.,"") //De/para filial do First
475
476 Default cEntidade := ""
477
478 Do Case
479 Case cAliasEnt == "SA1" //Cliente
480     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave //ALTERADO - RECEBEU PELO PDV
481 Case cAliasEnt == "SE4" //Condicao
482     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave
483 Case cAliasEnt == "SA3" //Vendedor
484     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave
485 Case cAliasEnt == "SB1" //Produto
486     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave
487
488 Case cAliasEnt == "SL1" //VENDAS
489     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave
490
491 Otherwise
492     cChaveRet := cEntidade +PadR(cFilFirst, nTamFil)+cChave
493 EndCase
494
495 Return cChaveRet
```

Framework/UpTables/STDUpData.prw

2405 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDUPDATA.CH"
4 #INCLUDE "TBICONN.CH"
5
6 STATIC _nMaxRegUp := 300
7
8 //=====
9 //          Situacoes do _SITUA
10 //
11 // " " - Base Errada, Registro Ignorado.
12 // "00" - Venda Efetuada com Sucesso
13 // "01" - Abertura do Cupom Nao Impressa
14 // "02" - Impresso a Abertura do Cupom
15 // "03" - Item Nao Impresso
16 // "04" - Impresso o Item
17 // "05" - Solicitado o Cancelamento do Item
18 // "06" - Item Cancelado
19 // "07" - Solicitado o Cancelamento do Cupom
20 // "08" - Cupom Cancelado
21 // "09" - Encerrado SL1 (Nao gerado SL4)
22 // "10" - Encerrado a Venda
23 //      Pode nao ter sido impresso o cupom
24 // "TX" - Foi Enviado ao Server (Pdv)
25 // "ER" - Erro ao envia ao server (Pdv)
26 // "EP" - Erro de processamento da venda (Pdv)
27 // "RE" - Ja foi feita nova tentativa de subir (Pdv)
28 // "RX" - Foi Recebido Pelo Server (Server)
29 // "OK" - Foi Processado no Server (Serevr)
30 // "OR" - Venda Salva como Orcamento
31 //      Enviar um OK ao Client que foi Processado
32 // "CP" - Recebido pela Central de PDV
33 // "RY" - Cancelamento Nfc-e
34 // "CX" - Cancelamento enviado first
35 // "C0" - Cancelamento a enviar first
36 // "65" - Gravação das tabelas para transmissão da NFC-e
37 //-----
38 // Status da transmissão por NFce Dll que não vão para retaguarda - interno TotvsPDV Mobile
39 // "D1" - NFce Dll - Contingencia off-line - Envia NFce em contingência e consulta Status da NFce para então enviar para retaguarda
40 // "D0" - NFce Dll - Contingencia on-line - Consulta Status da NFce para então enviar para retaguarda
41 // "I8" - NFce Dll - Inutilização não realizada - Tentar novamente - Se deu certo envia venda inutilizada para retaguarda
42 // "DC" - NFce Dll - Cancelamento on line realizado de Nota não enviada - envia o cancelamento novamente
43 // "DX" - NFce Dll - Cancelamento on line realizado de Nota enviada - consulta status da nfce antes de enviar a retaguarda
44 // "D7" - NFce Dll - Cancelamento offline - não enviado - envia o cancelamento em contingencia e depois consulta o status
45 // "D8" - NFce Dll - Cancelamento on line não realizado de nota enviada - tenta realizar o cancelamento e consulta o status
46 // "D9" - NFce - Cancelamento on line não realizado de nota NÃO enviada - tenta cancelar a nota e consulta o status
47 //=====
48
49
50 //-----
51 /*/{Protheus.doc} STDsalesForUp
52 Realiza a gravacao da venda no BackOffice

```

```

53
54 @param cEstacaoIni      Codigo da estacao
55
56 @return Nil
57 @obs
58 @sample
59 /*/
60 //-----
61 Function STDSalesForUp( cEstacaoIni )
62
63 Local aSL1      := {} // array com dados da SL1
64 Local aSL2      := {} // array com dados da SL2
65 Local aSL4      := {} // array com dados da SL4
66 Local lEncontrou := .T. // Se encontrou registro
67 Local nX        := 1 // Contador
68 Local cL1_NUM   := "" // Numero da venda encontrada
69 Local lHasConnect := .T. // Verifica se tem comunicacao
70
71 Default cEstacaoIni := "001"
72
73 cPdv := cEstacaoIni // Carrega numero do PDV
74
75 while lEncontrou .AND. ( nX <= _nMaxRegUp )
76
77     Sleep(1000)
78
79     aSL1      := {}
80     aSL2      := {}
81     aSL4      := {}
82     cL1_NUM := ""
83
84     aSL1 := STDSL1Create(@lEncontrou, @cL1_NUM) //Recebe o Número do Orçamento gerado
85
86     If lEncontrou
87         aSL2 := STDSL2Create(@lEncontrou, cL1_NUM) //Passa o Número do Orçamento gerado
88     EndIf
89
90     If lEncontrou
91         aSL4 := STDSL4Create(@lEncontrou, cL1_NUM) //Passa o Numero do Orçamento gerado
92     EndIf
93
94     If lEncontrou
95
96         lHasConnect := .T. // Inicia sempre com a premissa que tem conexao
97         STDUpdSaleLocal( aSL1      , aSL2 , aSL4 , cEstacaoIni,;
98                         cL1_NUM   , @lHasConnect )
99
100        If !lHasConnect
101            Exit
102        EndIf
103    ElseIf Len(aSL1) > 0 .AND. Len(aSL2) == 0
104        //Venda Incompleta sera marcada como 'ER'

```

```

105      cL1_NUM := aSL1[AScan(aSL1,{|x|x[1]="L1_NUM"})][2]
106
107      DbSelectArea("SL1")
108      DbSetOrder(1) //L1_FILIAL+L1_NUM
109      If DbSeek( xFilial("SL1") + cL1_NUM )
110
111          If RecLock( "SL1", .F. )
112              REPLACE SL1->L1_SITUA WITH "ER"
113              DbCommit()
114              MsUnLock()
115              STDLogCons( STR0019 + cL1_NUM + STR0020 )// "Venda: " ### " Incompleta gravada como Erro"
116          EndIf
117
118      EndIf
119
120      lEncontrou := .T. //Para Permanecer no laço no reinicio
121
122      //PARA OS CASOS QUE NAO SE GRAVA O SL4 E NAO EH PAGTO COM NCC NAO DEVO SUBIR A VENDA
123      ElseIf Len(aSL1) > 0 .AND. Len(aSL2) > 0 .AND. Len(aSL4) == 0
124          //Venda Incompleta sera marcada como 'ER'
125          cL1_NUM := aSL1[AScan(aSL1,{|x|x[1]="L1_NUM"})][2]
126
127          DbSelectArea("SL1")
128          DbSetOrder(1) //L1_FILIAL+L1_NUM
129          If DbSeek( xFilial("SL1") + cL1_NUM )
130              If SL1->L1_CREDITO > 0
131                  lEncontrou := .T.
132              Else
133                  If RecLock( "SL1", .F. )
134                      REPLACE SL1->L1_SITUA WITH "ER"
135                      DbCommit()
136                      MsUnLock()
137                      STDLogCons( STR0019 + cL1_NUM + STR0020 )// "Venda: " ### " Incompleta gravada como Erro"
138                  EndIf
139              EndIf
140          EndIf
141
142          lEncontrou := .T. //Para Permanecer no laço no reinicio
143
144      EndIf
145      Sleep(1000)
146
147      nX++
148
149 EndDo
150
151 //Sinaliza as vendas para serem re-enviadas
152 SL1->(DbSetOrder(9)) //L1_FILIAL+L1_SITUA+L1_PDV+L1_DOC
153
154 // Procura registro com erro de envio
155 // Procura registros erro de processamento
156 SL1->(DbSeek(xFilial("SL1")+"RE"))

```

```

157
158 Do While SL1->(!Eof() .AND. L1_FILIAL +L1_SITUA == xFilial("SL1")+"RE")
159     RecLock("SL1", .F.)
160     SL1->L1_SITUA := "EP"
161     SL1->(MsUnLock())
162     SL1->(DbSkip())
163     SL1->(DbSeek(xFilial("SL1")+"RE"))
164 EndDo
165
166 Return Nil
167
168
169 //-----
170 /*{Protheus.doc} STD_SL1Create
171 Busca Vendas para subir ao BackOffice
172
173 @param lEncontrou      Indica se encontrou a venda - Variavel usada por referencia
174
175 @return  aSL1          array com dados da SL1
176 @obs
177 @sample
178 */
179 //-----
180 Function STD_SL1Create( lEncontrou, cL1_NUM )
181
182 Local nCampos      := 0           // Quantidade de campos do filtro atual
183 Local aSL1         := {}         // Array do Alias
184 Local i            := 0           // Contador
185 Local lCentPDV     := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[1] , .F. ) // Eh Central de PDV
186 Local lFirst       := .F.        //Integração First
187 Local cIntegration  := ""         //Tipo de Integração
188 Local lSaveOrc     := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
189 Local cFieldName   := ""         // Armazena o nome do campo
190
191 Default lEncontrou := .F.
192 Default cL1_NUM    := ""
193
194 ParamType 0 var    lEncontrou      As Logical   Default   .F.
195 ParamType 1 var    cL1_NUM         As Character Default ""
196
197 lEncontrou := .F.
198
199 If ExistFunc("STFCfgIntegration")
200     cIntegration := STFCfgIntegration()
201 Endif
202
203 lFirst := cIntegration == "FIRST"
204
205 DbSelectArea("SL1")
206 SL1->( DbSetOrder(9) ) //L1_FILIAL+L1_SITUA+L1_PDV+L1_DOC
207
208 // Procura registro com erro de envio

```

```

209 // Procura registros erro de processamento
210 If !lEncontrou
211     lEncontrou := DbSeek(xFilial("SL1")+"EP")
212 EndIf
213
214 //Procura Vendas
215 If !lEncontrou
216     If lCentPDV
217         lEncontrou := SL1->( DbSeek(xFilial("SL1")+"CP") )
218     Else
219         lEncontrou := SL1->( DbSeek(xFilial("SL1")+"00") )
220     EndIf
221 EndIf
222
223 If !lEncontrou .AND. lFirst
224     lEncontrou := SL1->( DbSeek(xFilial("SL1")+"C0") ) //Cancelamento via first - reenvio
225 EndIf
226
227 //Verifica venda salva como orcamento
228 If !lEncontrou .AND. lSaveOrc
229     lEncontrou := SL1->( DbSeek(xFilial("SL1")+"OR") )
230 EndIf
231
232 If lEncontrou
233     cL1_NUM := SL1->L1_NUM
234
235     nCampos := FCount()
236     aSL1 := Array(nCampos)
237
238     For i := 1 To nCampos
239
240         cFieldName := FieldName(i)
241
242         /* Não manda o conteudo dos campos Log de Inclusao/Alteracao, pois senao
243         gera um erro na subida da venda por causa dos caracteres especiais.
244         Além disso, seu conteudo seria sobrescrito ao gravar na retaguarda */
245         If cFieldName $ "L1_USERLGI|L1_USERLGA"
246             aSL1[i] := { cFieldName, "" }
247         ElseIf cFieldName $ "L1_CONTCDC"
248             If !Empty(FieldGet(i)) .AND. Type(FieldGet(i))= "N"
249                 aSL1[i] := { cFieldName, PadL(Val(FieldGet(i)), TamSx3("L1_CONTCDC")[1], "0") }
250             Else
251                 aSL1[i] := { cFieldName, FieldGet(i) }
252             EndIf
253         Else
254             aSL1[i] := { cFieldName, FieldGet(i) }
255         EndIf
256
257     Next i
258
259     // Salva origem da venda
260     If SL1->L1_OPERACA != "C"

```

```

261      aSL1[AScan(aSL1,{|x|x[1]=="L1_NUMORIG"})][2] := SL1->L1_NUM
262 EndIf
263
264 If lCentPDV //Para buscar o orcamento pelo DOC e Serie e evitar vendas "DU"
265      aSL1[AScan(aSL1,{|x|x[1]=="L1_OPERACA"})][2] := ""
266 EndIf
267
268 //Apaga Possiveis mensagens de erro gravada como Log
269 //pois o campo será reaproveitado na retaguarda e devera estar vazio
270 If SL1->(FieldPos("L1_ERGRVBT") > 0)
271      aSL1[AScan(aSL1,{|x|x[1]=="L1_ERGRVBT"})][2] := ""
272 EndIf
273
274 EndIf
275
276 Return aSL1
277
278 //-----
279 /*/{Protheus.doc} STDSL2Create
280 Busca Vendas para subir ao BackOffice
281
282 @param lEncontrou      Indica se encontrou a venda - Variavel usada por referencia
283
284 @return aSL2           array com dados da SL2
285 @obs
286 @sample
287 */
288 //-----
289 Function STDSL2Create( lEncontrou, cL1_NUM)
290
291 Local aArea           := GetArea()      // Armazena alias corrente
292 Local nCampos         := 0             // Quantidade de campos do filtro atual
293 Local aSL2            := {}            // Array do Alias
294 Local i               := 0             // Contador
295 Local lSaveOrc        := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
296 Local xValue
297
298 Default lEncontrou := .F.
299 Default cL1_NUM := ""
300
301 If !Empty(cL1_NUM)
302     DbSelectArea("SL2")
303     DbSetOrder(1)//L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
304     If DbSeek(xFilial("SL2")+cL1_NUM)
305
306         lEncontrou := .T.
307         nCampos := FCount()
308         aSL2 := {Array(nCampos)}
309
310         For i := 1 To nCampos
311             aSL2[1][i] := FieldName(i)
312         Next i

```

```

313
314 While SL2->L2_FILIAL+SL2->L2_NUM == xFilial("SL2")+cL1_NUM .AND. !EOF()
315
316 //Traz registros deletados, porem so considera itens L2_VENDIDO = "S"
317 If (SL2->L2_VENDIDO == "S" .AND. SL2->L2_SITUA <> "05") .OR. ; // ignora Registros cancelados no orçamento
318 (lSaveOrc .AND. Empty(SL2->L2_VENDID0)) //Se salva como orcamento sobe item
319
320 AAdd(aSL2, Array(FCount()))
321
322 For i := 1 To nCampos
323
324 // 0 campo descricao pode ter caracter especial, nesse caso remove
325 If aSL2[1][i] == "L2_DESCRI" .AND. ExistFunc("LjRmvAcent")
326 xValue := LjRmvAcent( FieldGet(i) )
327 // Busque a explicacao no comentario do trecho "L1_USERLGI|L1_USERLGA"
328 ElseIf aSL2[1][i] $ "L2_USERLGI|L2_USERLGA"
329 xValue := ""
330 Else
331 xValue := FieldGet(i)
332 EndIf
333
334 aSL2[Len(aSL2)][i] := xValue
335 Next i
336 EndIf
337
338 SL2->(DbSkip())
339 EndDo
340 Else
341 lEncontrou := .F.
342 EndIf
343 Else
344 lEncontrou := .F.
345 EndIf
346
347 RestArea(aArea)
348
349 Return aSL2
350
351
352 //-----
353 /*{Protheus.doc} STDSL4Create
354 Busca Vendas para subir ao BackOffice
355
356 @param lEncontrou Indica se encontrou a venda - Variavel usada por referencia
357
358 @return aSL4 array com dados da SL4
359 @obs
360 @sample
361 */
362 //-----
363 Function STDSL4Create( lEncontrou, cL1_NUM )
364

```

```

365 Local   aArea      := GetArea()           // Armazena alias corrente
366 Local nCampos     := 0                   // Quantidade de campos do filtro atual
367 Local aSL4        := {}                  // Array do Alias
368 Local i           := 0                   // Contador
369
370 Default lEncontrou := .F.
371 Default cL1_NUM    := ""
372
373 If !Empty(cL1_NUM)
374     DbSelectArea("SL4")
375     DbSetOrder(1) //L4_FILIAL+L4_NUM+L4_ORIGEM
376     If DbSeek(xFilial()+cL1_NUM)
377
378         lEncontrou := .T.
379
380         nCampos := FCount()
381         aSL4 := {Array(nCampos)}
382
383         For i := 1 To nCampos
384             aSL4[1][i] := FieldName(i)
385         Next i
386
387         While SL4->( !EOF() ) .AND. SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + cL1_NUM
388             AAdd(aSL4, Array(nCampos))
389
390             For i := 1 To nCampos
391
392                 // Busque a explicacao no comentario do trecho "L1_USERLGI|L1_USERLGA"
393                 If aSL4[1][i] $ "L4_USERLGI|L4_USERLGA"
394                     aSL4[Len(aSL4)][i] := ""
395                 Else
396                     aSL4[Len(aSL4)][i] := FieldGet(i)
397                 EndIf
398
399             Next i
400
401             SL4->( DbSkip() )
402         EndDo
403
404     ElseIf SL1->L1_CREDITO > 0
405         lEncontrou := .T.
406     Else
407         lEncontrou := .F.
408     EndIf
409
410 Else
411     lEncontrou := .F.
412 EndIf
413
414 RestArea(aArea)
415
416 Return aSL4

```

```

417
418 //-----
419 /*{Protheus.doc} STDUpdSaleLocal
420 Gera valor dos campos do sl1 sl2 e sl4
421 Atualiza situacao das tabelas locais que ja subiram para o server
422
423 @param aSL1      Array da venda SL1
424 @param aSL2      Array da venda SL2
425 @param aSL4      Array da venda SL4
426 @param cEstacaoIni Estacao
427
428 @return Nil
429 @obs
430
431 Situacoes do cStatusNum :
432 "OK|"+L1_NUM    = Venda subiu com sucesso
433 "BX|"+L1_NUM    = Tentativa de duplicar. A venda ja consta no servidor
434 "ERRO"         = Erro ao subir venda
435
436 @sample
437 */
438 //-----
439 Function STDUpdSaleLocal( aSL1      , aSL2 , aSL4 , cEstacaoIni, ;
440                          cL1_NUM   , lHasConnect)
441
442 Local aArea           := GetArea()    // Armazena alias corrente
443 Local cEstacao        := ""          // Estacao
444 Local aUpdSL1         := {}          // Array para Atualizacao da SL1
445 Local cStatusNum      := ""          // Recebe Status + Numero da venda
446 Local cNum            := ""          // Numero da venda da cStatusNum apos retirar caracteres especiais
447 Local cNumSaleConf    := ""          // Recebe numero da venda confirmada no server
448 Local lContinua      := .F.         // Controle de execucao
449 Local lComCPDV        := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[2] , .F. ) // Usa comunicacao com a central
450 Local lFirst          := .F.
451 Local cIntegration    := ""
452 Local cRotinaExec     := ""
453 Local nCodRet         := 0
454 Local lSaveOrc        := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
455 Local cDescError      := ""          // Guarda descricao do erro do componente de comunicação
456 Local lHostError      := .F.         // Controla se houve erro na execucao do host
457
458 Default lHasConnect   := .T.
459 Default aSL1          := {}
460 Default aSL2          := {}
461 Default aSL4          := {}
462 Default cEstacaoIni   := ""
463 Default cL1_NUM       := ""
464
465
466 /*
467     Adicionado RecLock devido a um problema no cancelamento do cupom.
468     Apos incluir uma venda, e nao tiver dado tempo para a gravacao do pacote no server,

```

```

469     o cancelamento nao era feito no server, causando problemas de sincronizacao de base.
470 */
471
472 If ExistFunc("STFCfgIntegration")
473     cIntegration := STFCfgIntegration()
474 Endif
475
476 lFirst := cIntegration == "FIRST" .AND. ExistFunc("STDUpFirstSale")
477
478 //voltar a workarea sl1-> (9)
479
480 DbSelectArea( "SL1" )
481 If !Empty(cL1_NUM) .AND. !(AllTrim(cL1_NUM) == AllTrim(SL1->L1_NUM))
482     SL1->( DbSetOrder(1) )
483     SL1->( DbSeek(xFilial("SL1") + cL1_NUM) )
484 Endif
485
486 If RecLock( "SL1", .F. )
487     LjGrvLog( cL1_NUM, "Antes do envio da venda para a retaguarda" )
488     LjGrvLog( cL1_NUM, "Num DOC da Venda "+SL1->L1_DOC )
489     // Realizado comunicacao via componente de comunicacao para Gravar a venda
490     If !lFirst
491         If lComCPDV
492             //Se usa a Central de PDV executa outra rotina para evitar erro no componente
493             cRotinaExec := "_GeraL1L2L4"
494         Else
495             cRotinaExec := "GeraL1L2L4"
496         Endif
497         lContinua := STBRemoteExecute( cRotinaExec, { aSL1, aSL2, aSL4, cEstacaoIni} , NIL , .F. , ;
498             @cStatusNum, /*cType*/ , /*cKeyOri*/ , @nCodRet , ;
499             @cDescError)
500     Else
501         lContinua := STDUpFirstSale( aSL1, aSL2, aSL4, cEstacaoIni, @cStatusNum )
502     Endif
503
504     LjGrvLog( cL1_NUM, "Retorno logico do envio da venda",lContinua)
505     LjGrvLog( cL1_NUM, "Retorno string do envio da venda",cStatusNum)
506     LjGrvLog( cL1_NUM, "Codigo retorno da execucao na retaguarda",nCodRet)
507
508     // Se retornar esses codigos siginifica que a retaguarda ta fora, provavelmente tava online (pq se nao nem chegaria aqui )
509     // e depois perdeu, tratamos todos esses codigos, mais o correto é o frame nos retornar apenas -105 que eh significa
510     // sem comu , porem vimos que para WS pode retornar tb -107 e -104 que esta errado, porem tratamos mesmo assim
511     lHasConnect := !(nCodRet == -105 .OR. nCodRet == -107 .OR. nCodRet == -104)
512
513     // Verifica erro de execucao por parte do host
514     //-103 : Ocorreu um erro na execução da funcionalidade
515     //-106 : 'Não foi possível deserializar os parametros (JSON)
516     lHostError := (nCodRet == -103 .OR. nCodRet == -106)
517
518     If ValType( cStatusNum ) == "C" .AND. lHasConnect .AND. !lHostError //tem comunic com a Ret e nao teve erro
519         // Verifica se a venda subiu com Sucesso "OK"
520         If Left( cStatusNum, 2 ) == "OK"

```

```

521     cNum := Substr( cStatusNum, 4, 6 )
522     If !Empty( cNum )
523         // Realizado comunicacao via componente de comunicacao
524         // Confirma se a venda foi e criada retornando o numero da mesma
525         If !lFirst .AND. !lSaveOrc
526             If lComCPDV
527                 cRotinaExec := "_ConfL1L2L4" //Se usa a Central de PDV executa outra rotina para evitar erro no componente
528             Else
529                 cRotinaExec := "ConfL1L2L4"
530             EndIf
531             lContinua := STBRemoteExecute(cRotinaExec, { cNum , cEstacaoIni } , NIL, .F. , @cNumSaleConf )
532         Else
533             cNumSaleConf := cNum
534         EndIf
535
536         LjGrvLog( cL1_NUM, "Ret da confirmacao da vend",lContinua)
537         LjGrvLog( cL1_NUM, "Ret string da confirmacao da venda",cStatusNum)
538
539         DbSelectArea( "SL1" )
540
541         If ValType( cNumSaleConf ) == "C"
542
543             If cNum <> cNumSaleConf
544                 // Erro ao confirmar a gravacao do Orcamento "
545                 STDLogCons( STR0002 + SL1->L1_NUM + "." ) //"Erro ao confirmar a gravacao do Orcamento "
546                 LjGrvLog( SL1->L1_NUM,"Orc diferente "+cNum+" e cNumSaleConf "+cNumSaleConf)
547             Else
548
549                 // Verifica se houve o cancelamento da Venda antes de mudar o _SITUA
550                 // Se Houve, sera enviado o cancelamento via SLI
551                 If SL1->L1_STORC == "C" .AND. !lFirst
552                     aUpdSL1 := { { "L1_NUMORIG", cNumSaleConf },,
553                                 { "L1_SITUA" , "07" } } // "07" - L1_SITUA padrao para Venda Cancelada
554                 Else
555                     aUpdSL1 := { { "L1_NUMORIG", cNumSaleConf } , ;
556                                 { "L1_SITUA", IIF(!lFirst .OR. SL1->L1_SITUA <> "C0" ,"TX", "CX") } } // "TX" - Foi Enviado ao Server / "CX" -
557             Cancelamento enviado ao server
558             EndIf
559
560             aEval( aUpdSL1 , { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
561
562             STDLogCons( STR0003 + SL1->L1_NUM + STR0004 ) //"Orcamento gravado com sucesso."
563             LjGrvLog( SL1->L1_NUM,"Orcamento gravado com sucesso.")
564         EndIf
565     EndIf
566 Else
567     If Left( cStatusNum, 2 ) == "BX"
568         aUpdSL1 := { { "L1_SITUA", "DU" } } // "DU" - Orcamento duplicado no server
569         aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
570         LjGrvLog( SL1->L1_NUM, "Retorno DU na estacao")
571     ElseIf Left( cStatusNum, 2 ) == "ER"

```

```

572      LjGrvLog( SL1->L1_NUM, "Retornou erro no processamento da venda grava como ER. " + cStatusNum )
573      aUpdSL1 := { { "L1_SITUA", "ER" } } // "ER" - Erro ao gravar no server
574      If SL1->L1_SITUA <> "TX"
575          aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
576      Else
577          LjGrvLog( SL1->L1_NUM, "Venda ja subiu - SL1->L1_SITUA:", SL1->L1_SITUA)
578      EndIf
579      If SL1->(FieldPos("L1_ERGRVBT") > 0)
580          aUpdSL1 := { { "L1_ERGRVBT", Left(cStatusNum,TamSX3("L1_ERGRVBT")[1]) } } // "ER" - Erro ao gravar no server
581          aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
582      EndIf
583  Else
584      //Se tiver como EP significa que eh a segunda vez que da erro na subida, ai fica ER
585      If SL1->L1_SITUA == "EP"
586          LjGrvLog( SL1->L1_NUM, "Retorno inesperado novamente grava como ER")
587          aUpdSL1 := { { "L1_SITUA", "ER" } } // "ER" - Erro ao gravar no server
588      Else
589          LjGrvLog( SL1->L1_NUM, "Retorno inesperado grava como RE")
590          aUpdSL1 := { { "L1_SITUA", "RE" } } // "RE" - Erro ao gravar no server
591      EndIf
592      aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
593  EndIf
594 EndIf
595 Else
596     // Se houve erro na execucao do host
597     If lHostError
598         aUpdSL1 := { { "L1_SITUA", "ER" } } // "ER" - Erro ao gravar no server
599         LjGrvLog( SL1->L1_NUM, "Nao conseguiu gravar venda. Erro componente de comunicação,cStatusNum)
600         aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
601         If ValType(cDescError) == "C" .AND. SL1->(FieldPos("L1_ERGRVBT") > 0)
602             aUpdSL1 := { { "L1_ERGRVBT", Left(cDescError,TamSX3("L1_ERGRVBT")[1]) } } // "ER" - Erro ao gravar no server
603             If SL1->L1_SITUA <> "TX"
604                 aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
605             Else
606                 LjGrvLog( SL1->L1_NUM, "Venda ja subiu - SL1->L1_SITUA:", SL1->L1_SITUA)
607             EndIf
608         EndIf
609     ElseIf lHasConnect // Caso tenha conexao nesse momento e msm assim nao subiu grava como "ER"
610         If SL1->L1_SITUA == "EP"
611             aUpdSL1 := { { "L1_SITUA", "ER" } } // "ER" - Erro ao gravar no server
612             LjGrvLog( SL1->L1_NUM, "Nao conseguiu na segunda, grava ER",cStatusNum)
613         Else
614             LjGrvLog( SL1->L1_NUM, "Nao foi possivel gravar o Orcamento no servidor Será marcado para reprocessar.",cStatusNum)
615             //Grava venda como RE (Reenviar) pois houve erro ao enviar ao server
616             aUpdSL1 := { { "L1_SITUA", "RE" } } // "RE" - Erro ao enviar ao server
617         EndIf
618         aEval( aUpdSL1, { |x| FieldPut( FieldPos( x[1] ), x[2] ) } )
619     Else
620         LjGrvLog( SL1->L1_NUM, "No momento de mandar a venda nao conseguiu conectar")
621     EndIf
622 EndIf
623 Else

```

```

624     STDLogCons( STR0011 + SL1->L1_NUM + "." ) //"Erro ao confirmar a gravacao do Orcamento"
625 EndIf
626
627 SL1->( DbCommit() )
628 SL1->( MsUnlock() )
629
630 RestArea(aArea)
631
632 Return Nil
633
634
635 //-----
636 /*{Protheus.doc} STDCancSales
637 Efetua a gravacao do cancelamento no server
638
639 @param cEstacaoIni      Codigo da estacao
640
641 @return Nil
642 @obs
643 @sample
644 */
645 //-----
646
647 Function STDCancSales( cEstacaoIni )
648
649 Local aArea      := GetArea()// Armazena alias corrente
650 Local cOrcamentos := ""      // Concatenacao de vendas canceladas
651 Local cNumOrig   := ""      // Numero de origem
652 Local nOrcs     := 1        // Numero orcamento
653 Local cEstacao  := ""      // Estacao
654 Local aOrcs     := {}       // Array de orcamentos
655 Local aRet      := {}       // Array de retorno
656 Local lContinua := .F.      // Controle de execucao
657 Local lComCPDV  := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[2] , .F. ) // Usa comunicacao com a central
658 Local cExecFunc := "STDCancRec" //função de cancelamento a ser executada
659 Local aAuxLIMsg := []
660 Local cAuxSeek  := ""
661 Local aSL1      := {}
662 Local lSaveTabOk := .F.
663 Local aDupDel    := {}      //Array de registros duplicados a deletar
664 Local lCentPDV   := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[1] , .F. ) // Eh Central de PDV
665 Local aParam     := {}
666 Local nI         := 0       //Variável auxiliar
667 Local cSerNf     := ""      //Serie nao fiscal
668 Local lUseSAT    := LjGetStation("LG_USESAT")
669 Local lSerNFe    := .F.
670 Local lFunPdv    := ExistFunc("STFPdvOn")
671 Local lPdvOn     := .F.
672 Local lRPS       := .F.     //Cancelamento de Venda com RPS
673 Local nPosaRet   := 0       //Posição do aRet
674
675 Default cEstacaoIni := "001"

```

```

676
677
678 cEstacao := Space(TamSX3("LI_ESTACA0")[1])
679
680 DbSelectArea("SLI")
681 SLI->(DbSetOrder( 1 )) //LI_FILIAL+LI_ESTACA0+LI_TIPO
682 If SLI->(DbSeek( xFilial("SLI") + cEstacao + "CAN" ))
683
684     While !SLI->(EOF()) .AND. (SLI->LI_FILIAL + SLI->LI_ESTACA0 + SLI->LI_TIPO) == (xFilial("SLI") + cEstacao + "CAN")
685
686         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Inicio do processo de subida do cancelamento para Retaguarda SLI(LI_TIPO = CAN) -> LI_MSG:",SLI->LI_MSG)
687
688         If !Empty(SLI->LI_MSG)
689
690             aAuxLIMsg := StrTokArr( RTrim(SLI->LI_MSG), "|" ) //L1_NUMORIG|L1_DOC|L1_PDV
691
692             If Len(aAuxLIMsg) >= 3
693
694
695                 aAuxLIMsg[3] := PADR(aAuxLIMsg[3],TamSX3("L1_PDV")[1]) //Ajusta tamanho do conteudo do campo L1_PDV, quando armazenado na ultima posicao do campo LI_MSG, fica
com tamanho total do campo. Nao pode ser tratado na gravacao, se adicionado "|" para limitar tamanho, quando convertido para array vai retornar que tem 4 posicoes
696
697                 lSerNFe := Len(aAuxLIMsg) >= 5 .and. !Empty(aAuxLIMsg[5])
698                 lRPS := Len(aAuxLIMsg)>= 6 .AND. !Empty(aAuxLIMsg[6]) //Cancelamento de Venda com RPS
699
700                 if lSerNFe
701
702                     If lRPS //Cancelamento de Venda com RPS
703                         SL1->( DbSetOrder(1) )//L1_FILIAL+L1_NUM
704                         aAuxLIMsg[6] := PADR(aAuxLIMsg[6],TamSX3("L1_NUM")[1])
705                         cAuxSeek := xFilial("SL1") + aAuxLIMsg[6]
706                     Else
707                         //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
708                         SL1->( DbSetOrder(2) )
709                         aAuxLIMsg[5] := PADR(aAuxLIMsg[5],TamSX3("L1_SERIE")[1])
710                         cAuxSeek := xFilial("SL1") + aAuxLIMsg[5] + aAuxLIMsg[2] + aAuxLIMsg[3]
711                     Endif
712
713                 else
714                     If Len(aAuxLIMsg)>= 6 .AND. !Empty(aAuxLIMsg[6])
715                         //L1_FILIAL+L1_NUM
716                         SL1->( DbSetOrder(1) )
717                         cAuxSeek := xFilial("SL1") + aAuxLIMsg[6]
718                     Else
719                         //L1_FILIAL+L1_PDV+L1_DOC
720                         SL1->( DbSetOrder(8) )
721                         cAuxSeek := xFilial("SL1") + aAuxLIMsg[3] + aAuxLIMsg[2]
722                     Endif
723                 endif
724
725                 LjGrvLog("SUBIDA_CANCELAMENTO" ,"SL1 sera posicionada para validar o status da venda(L1_SITUA). Busca por L1_FILIAL+L1_PDV+L1_DOC:",cAuxSeek)
726

```

```

727 DbSelectArea("SL1")
728 If !lUseSAT .AND. Len(aAuxLIMsg) >= 4 .AND. !Empty(aAuxLIMsg[4])
729     //So entra no IF se for uma venda de vale credito, pois assim pesquisa na SL1 pelo campos L1_DOCPED e L1_SERPED
730
731     cSerNf := PadR( STDGetSrNF(cEstacaoIni,"LG_SERNFIS"), SL1->(TamSx3("L1_SERPED"))[1] )
732
733     SL1->(DbSetOrder(11)) //L1_FILIAL+L1_SERPED+L1_DOCPED
734     If SL1->(DbSeek( xFilial("SL1")+cSerNf+aAuxLIMsg[4] )) .AND. AllTrim(SL1->L1_PDV) == AllTrim(aAuxLIMsg[3])
735         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Registro localizado na SL1. L1_DOCPED:",SL1->L1_DOCPED)
736         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Registro localizado na SL1. L1_SITUA",SL1->L1_SITUA)
737     Else
738         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao localizou o registro na SL1")
739         SLI->( DbSkip() )
740         Loop
741     EndIf
742 Else
743
744     If SL1->(DbSeek( cAuxSeek ))
745         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Registro localizado na SL1. L1_NUMORIG(Conteudo do campo L1_NUM da retaguarda que faz referencia a venda no
PDV):",SL1->L1_NUMORIG)
746         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Registro localizado na SL1. L1_SITUA",SL1->L1_SITUA)
747     Else
748         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao localizou o registro na SL1")
749         SLI->( DbSkip() )
750         Loop
751     EndIf
752 EndIf
753
754 //Quando L1_SITUA = 00|CP , nao processa o cancelamento, aguarda processo de subida da venda
755 If SL1->L1_SITUA $ "00|CP"
756     LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Aguardando venda subir para a retaguarda(L1_SITUA=00|CP)",cAuxSeek)
757     SLI->( DbSkip() )
758     Loop
759 EndIf
760
761 If lFunPdv
762     lPdvOn := STFPdvOn(SL1->L1_ESTACAO)
763 EndIf
764
765 If Empty(SL1->L1_DOC) .AND. !Empty(SL1->L1_DOCPED)
766     //So entra no IF se for uma venda de vale credito
767     //Se for uma venda de vale credito, o sistema so ira cancelar a venda na retaguarda se a venda ja foi processada pelo GrvBatch
768
769     lContinua := STBRemoteExecute( "STDCONGRVB" , { aAuxLIMsg[3], aAuxLIMsg[2], .T., SL1->L1_SERPED, aAuxLIMsg[4] } , NIL, .F. , @aRet )
770     If !(Len(aRet) >= 1 .AND. ValType(aRet[1]) == "L" .AND. aRet[1] == .T.)
771         lContinua := .F.
772         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Aguardando Execucao de LjGrvBatch(L1_SITUA!=OK) na Retaguarda",cAuxSeek)
773         SLI->( DbSkip() )
774         Loop
775     EndIf
776 Else
777     //Quando Central de PDV, e não subiu o LjGrvBatch na retaguarda, nao processa o cancelamento

```

```

778         If lCentPDV .AND. !lPdvOn
779             lContinua := STBRemoteExecute( "STDCONGRVB" , { aAuxLIMsg[3], aAuxLIMsg[2], , , , iif(lSerNFe,aAuxLIMsg[5], Nil) } , NIL, .F. , @aRet )
780             If !(Len(aRet) >= 1 .AND. ValType(aRet[1]) == "L" .AND. aRet[1] == .T.)
781                 lContinua := .F.
782             EndIf
783             If !lContinua
784                 LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Aguardando Execucao de LjGrvBatch(L1_SITUA!=OK) na
Retaguarda",cAuxSeek)
785                 SLI->( DbSkip() )
786                 Loop
787             EndIf
788             //Atualização de L1_MSG:
789             //Na central, o L1_NUMORIG inicialmente é o L1_NUM do PDV. Após o LjGrvBatch, o L1_NUMORIG passará a ser o L1_NUM da Retaguarda.
790             If SL1->L1_NUMORIG <> aAuxLIMsg[1]
791                 nI := At( "|", SLI->LI_MSG )
792                 RecLock( "SLI", .F. )
793                 SLI->LI_MSG := SL1->L1_NUMORIG + Iif(nI > 0,Substr(SLI->LI_MSG,nI),"")
794                 SLI->( MsUnlock() )
795                 aAuxLIMsg[1] := SL1->L1_NUMORIG //Restaura novamente
796             EndIf
797         EndIf
798     EndIf
799
800     cNumOrig      := aAuxLIMsg[1] //Subst(SLI->LI_MSG,1,TamSX3("L1_NUMORIG")[1])
801
802     //Quando cancelamento ocorreu antes de subir a venda, nao possui NUMORIG na SLI, busca informacao da SL1
803     If !lPdvOn .OR. (lCentPDV .AND. lPdvOn)
804         If Empty(cNumOrig)
805             If !Empty(SL1->L1_NUMORIG)
806                 cNumOrig := SL1->L1_NUMORIG
807                 LjGrvLog("SUBIDA_CANCELAMENTO" ,"Cancelamento ocorreu antes de subir a venda para retaguarda(SLI->LI_MSG sem NUMORIG). Atualizado NUMORIG com
dados da SL1->L1_NUMORIG:",cNumOrig)
808             Else
809                 LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Nao possui informacao NUMORIG para cancelamento na Retaguarda.
Verifique o Registro da tabela SL1 do PDV. SL1->L1_NUM",SL1->L1_NUM)
810                 SLI->( DbSkip() )
811                 Loop
812             EndIf
813         EndIf
814     Else
815         cNumOrig := SL1->L1_NUM
816     EndIf
817
818
819     If cNumOrig $ cOrcamentos
820         //Item duplicado sera deletado da SLI
821         AADD(aDupDel,{SLI->(Recno()),cNumOrig,cAuxSeek} )
822     Else
823
824         If Empty(cOrcamentos)
825             cOrcamentos := cNumOrig
826         Else

```

```

827         cOrcamentos += "|" + cNumOrig
828     EndIf
829
830     // Adiciona Recno na primeira posicao para atualizacao
831     // do SLI e Numero do Orcamento na Segunda posicao e a chave de pesquisa para atualizar SITUA quando subir
832     AADD(a0rcs,{SLI->(Recno()),cNumOrig,cAuxSeek,lSerNFe,lRPS} )
833     n0rcs++
834 EndIf
835
836 EndIf
837
838 Else
839     LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Sem conteudo para identificar o cancelamento. Conteudo Esperado:
L1_NUMORIG|L1_DOC|L1_PDV LI_MSG.SLI->LI_HORA:",LI_HORA)
840 EndIf
841
842     SLI->(DbSkip())
843 EndDo
844
845 If n0rcs > 1
846
847     // Realizado comunicacao via componente de comunicacao
848     // Efetua a gravacao do cancelamento no server
849     If lComCPDV
850         //Se usa Central de Pdv Cancela a Venda primeiro na Central e a Central cancela na Retaguarda
851         cExecFunc := "_STDCCancRec"
852     EndIf
853
854     If Len(aAuxLIMsg) > 3 .and. !Empty(aAuxLIMsg[4])
855         lContinua := STBRemoteExecute( cExecFunc , { cOrcamentos, cEmpAnt, cFilAnt, aAuxLIMsg[4] } , NIL, .F. , @aRet, , , , (lPdvOn .AND. !lCentPDV) )
856     Else
857         lContinua := STBRemoteExecute( cExecFunc , { cOrcamentos, cEmpAnt, cFilAnt } , NIL, .F. , @aRet, , , , (lPdvOn .AND. !lCentPDV))
858     EndIf
859
860     If lContinua .AND. (Len(aRet) > 0)
861
862         For n0rcs:=1 To Len(a0rcs)
863
864             //Verifica se foi cancelado
865             nPosaRet := Ascan(aRet,{|x| Trim(x[1]) == a0rcs[n0rcs][2]})
866
867             If nPosaRet > 0
868
869                 //Atualiza SL1->L1_SITUA para sinalizar que cancelamento foi realizado na retaguarda
870                 DbSelectArea("SL1")
871
872                 If a0rcs[n0rcs][4] //lSerNFe
873                     If a0rcs[n0rcs][5] //lRPS CANCELAMENTO DE VENDA RPS
874                         SL1->( DbSetOrder(1) )           //L1_FILIAL+L1_NUM
875                     Else
876                         SL1->( DbSetOrder(2) )           //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
877                     Endif

```

```

878 Else
879     SL1->( DbSetOrder(8) )      //L1_FILIAL+L1_PDV+L1_DOC
880 Endif
881
882 If !lPdvOn .OR. (lPdvOn .AND. lCentPDV)
883     If SL1->(DbSeek( aOrcs[nOrcs][3] )) .AND. SL1->L1_SITUA <> '00'
884         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Atualizacao Status(L1_SITUA=07): Cancelado na Retaguarda",SL1->L1_NUM)
885
886         If lCentPDV
887             aSL1 := {"L1_DOC", "",{"L1_SERIE", "",{"L1_SITUA" , "07"},{"L1_STORC" , "C"}}
888         Else
889             aSL1 := {"L1_SITUA" , "07"},{"L1_STORC" , "C"}}
890         EndIf
891
892         lSaveTabOk := STFSaveTab( "SL1" , aSL1 )
893
894         If lSaveTabOk .AND. aOrcs[nOrcs][5] .AND. !Empty(aRet[nPosaRet][2]) //Cancelamento de venda RPS
895             lSaveTabOk := STDAItSLX(SL1->L1_FILIAL+SL1->L1_PDV+SL1->L1_DOCRPS, aRet[nPosaRet][2])
896         EndIf
897
898         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Atualizacao Status(L1_SITUA=07): Cancelado na Retaguarda (.T. ou .F.):",lSaveTabOk)
899     Else
900         LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao localizou o registro na SL1 para atualizacao de Status(L1_SITUA=07): Cancelado na Retaguarda",SL1->L1_NUM)
901         lSaveTabOk := .F.
902     EndIf
903 Else
904     lSaveTabOk := .T.
905 EndIf
906
907 //Remove Registro de pedido de Cancelamento
908 If lSaveTabOk
909     SLI->(DbGoto(aOrcs[nOrcs][1]))
910     RecLock("SLI",.F.)
911     SLI->(DbDelete())
912     SLI->(MsUnlock())
913     LjGrvLog("SUBIDA_CANCELAMENTO" ,"Foi processado o cancelamento na retaguarda. L1_NUM(Retaguarda):",aOrcs[nOrcs][2])
914
915     //Se CentralPdv e possui orcamento, limpa orcamento para reutilizacao
916     If lCentPDV .AND. !lPdvOn
917         If SL1->L1_SITUA = "07" .And. SL1->L1_STORC = "C" .And. SL1->L1_OPERACA == "C"
918             aParam := {SL1->L1_NUM, SL1->L1_PDV}
919             FR271HArq(aParam,,.T.)
920             MsExecAuto( {|a,b,c,d,e,f,g| LJ140EXC(a,b,c,d,e,f,g)}, "SL1", /*nReg*/, 5, /*aReserv*/, .T., SL1->L1_FILIAL, SL1->L1_NUM )
921         EndIf
922     EndIf
923 Else
924     LjGrvLog("SUBIDA_CANCELAMENTO" ,"Foi processado o cancelamento na retaguarda, mas nao foi possivel atualiza SL1 do PDV.",aOrcs[nOrcs][2])
925 EndIf
926
927 STDLogCons(STR0003 + aOrcs[nOrcs][2] + STR0012)      //" cancelado no server"
928 Else
929     LjGrvLog("SUBIDA_CANCELAMENTO" ,"Não foi processado o cancelamento na retaguarda. L1_NUM(Retaguarda):",aOrcs[nOrcs][2])

```

```

930          STDLogCons(STR0003 + aOrcs[nOrcs][2] + STR0013)          //" aguardando processar a venda para realizar cancelamento no server."
931      EndIf
932
933      Next nOrcs
934
935      Else
936          LjGrvLog("SUBIDA_CANCELAMENTO" ,"Nao foi processado o cancelamento. Motivo: Nao foi possivel realizar a comunicacao com o Server da Retaguarda",cExecFunc)
937          STDLogCons(STR0014) //"Erro de comunicacao com servidor"
938      EndIf
939
940      Else
941          LjGrvLog("SUBIDA_CANCELAMENTO" ,"Fim do processo de cancelamento. Nenhum registro foi enviado para cancelamento na Retaguarda.")
942      EndIf
943
944      If Len(aDupDel) > 0 //Remove Registro de pedido de Cancelamento duplicados
945
946          For nOrcs := 1 To Len(aDupDel)
947              SLI->(DbGoto(aDupDel[nOrcs][1]))
948              RecLock("SLI",.F.)
949              SLI->(DbDelete())
950              SLI->(MsUnlock())
951              LjGrvLog("SUBIDA_CANCELAMENTO" ,"Foi deletado item duplicado: ",aDupDel[nOrcs][2])
952          Next nOrcs
953
954      EndIf
955
956  EndIf
957
958  RestArea(aArea)
959
960  Return Nil
961
962  //-----
963  /*{Protheus.doc} STDCancRec
964  Efetua a gravacao do cancelamento
965
966  @param cEstacaoIni      Codigo da estacao
967
968  @param cRegistros      String co registros a cancelar
969  @param cEmp            codigo da Empresa
970  @param cFil            Codigo da Filial
971
972  @return  aRetDados - Daods do cancelamento
973  @obs
974  @sample
975  /*/
976  //-----
977  Function STDCancRec( cRegistros, cEmp, cFil, cNFisCanc, cSerie )
978
979  Local aOrcs          := {} // Array de orcamentos
980  Local nCont          := 1  // Variavel de controle
981  Local nQuant         := 0  // Qtd de orcamentos

```

```

982 Local nRet      := 1      // Retorno
983 Local aRetDados  := {}     // Array de retorno
984 Local lCentPDV   := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[1] , .F. ) // Eh Central de PDV
985 Local lAltEmp    := .F.
986 Local aSM0       := {}
987 Local cNumEmp    := ""
988 Local cL1DOCRet  := ""     // Retorna o numero do DOC da Ret, isso é necessário para atualizar o SLX
989                                     // pq qdo é venda RPS o PDV não pode gravar o L1_DOC e alem disso quando a venda sobe,
990                                     // a RET grava outro sequencial , por isso quando for cancelar é necessário gravar o doc correto no campo SLX
991
992
993 Default cRegistros := ""    // Registros a cancelar
994 Default cEmp       := ""    // codigo da Empresa
995 Default cFil       := ""    // Codigo da Filial
996 Default cNFisCanc  := ""    // DOC cancelamento SAT
997 Default cSerie     := ""
998
999 a0rcs := StrToKarr(cRegistros,"|")
1000 nQuant := Len(a0rcs)
1001
1002 LjGrvLog("STDCancRec","Registros a Cancelar", a0rcs)
1003 LjGrvLog("STDCancRec","Doc(SAT) para cancelar", cNFisCanc)
1004
1005 //-- Ao utilizar com NF-e devido ao processo da parte do faturamento vem desposicionado ao ser chamado pela integracao
1006 if ( AllTrim(SM0->M0_CODIGO) + AllTrim(SM0->M0_CODFIL) ) <> (cEmp + cFil)
1007
1008     LjGrvLog("STDCancRec","Ajustando SM0, estava desposicionada!", cNFisCanc)
1009
1010     lAltEmp := .T.
1011
1012     aSM0 := SM0->( GetArea() )
1013     if !SM0->( DbSeek( cEmp + cFil ) )
1014         LjGrvLog("STDCancRec","SM0 Nao encontrada!", cNFisCanc)
1015         nQuant := 0 //-- Zero para não entrar no loop e sair da funcao
1016     endif
1017
1018     cEmpAnt := SM0->M0_CODIGO
1019     cFilAnt := AllTrim(SM0->M0_CODFIL)
1020     cNumEmp := cEmpAnt + cFilAnt
1021
1022 endif
1023
1024 While nCont <= nQuant
1025
1026     //Deixar antes do FRTEclusa
1027     If lCentPDV // Eh Central de PDV
1028         //Se for central de PDV replica o cancelamento para a Retaguarda gerando SLI na Central
1029         If STDCSRequestCancel( a0rcs[nCont] )
1030             LjGrvLog("STDCancRec","Central de PDV recebeu o cancelamento(SLI)", a0rcs[nCont])
1031             nRet := 0
1032         Else
1033             LjGrvLog("STDCancRec","Central de PDV NAO recebeu o cancelamento(SLI)", a0rcs[nCont])

```

```

1034     EndIf
1035 Else
1036     nRet := FRTE exclusiva( a0rcs[nCont], cNFisCanc, @cL1D0CRet )
1037 EndIf
1038
1039 /*
1040     Retornos:  (0)Cancelou,
1041                (2)Nao Existe
1042                (3)Venda nao foi processada pelo GrvBatch
1043 */
1044
1045 If nRet == 0 .OR. nRet == 2 .OR. nRet == 3
1046     //Retorna Vendas Canceladas para atualizar SLI do PDV.
1047     //Somente gera SLI quando envia a venda para no server, quando não localiza a venda limpa SLI para evitar looping infinito
1048     If nRet <> 3
1049         AADD( aRetDados,{a0rcs[nCont], cL1D0CRet} )
1050     EndIf
1051 EndIf
1052
1053 EndIf
1054
1055     nCont++
1056 EndDo
1057
1058 LjGrvLog("STDCancRec","Retorno", nRet)
1059 LjGrvLog("STDCancRec","Dados Retornados", aRetDados)
1060
1061 if lAltEmp
1062     RestArea(aSM0)
1063
1064     cEmpAnt := cEmp
1065     cFilAnt := cFil
1066     cNumEmp := cEmpAnt + cFilAnt
1067 endif
1068
1069 Return aRetDados
1070
1071 //-----
1072 /*/{Protheus.doc} STDLogCons
1073 Enviado o Log para o Console
1074
1075 @param cMessage      Codigo da estacao
1076
1077 @return Nil
1078 @obs
1079 @sample
1080 */
1081 //-----
1082 Static Function STDLogCons( cMessage )
1083
1084 Default cMessage := ""
1085

```

```

1086 ParamType 0 Var      cMessage  As Character  Default  ""
1087
1088 Conout(cMessage)
1089 LjGrvLog("STDUpData", cMessage)
1090
1091 Return Nil
1092
1093
1094 //-----
1095 /*/{Protheus.doc} STDRecEstSale
1096 Grava o Estorno da Venda no server
1097
1098 @return Nil
1099 @obs
1100 @sample
1101 */
1102 //-----
1103 Function STDRecEstSale()
1104
1105 Local  aArea      := GetArea()      // Armazena alias corrente
1106 Local  nI         := 0              //Variavel de apoio
1107 Local  aMBZ       := {}             //Array com os campos do MBZ
1108 Local  aRet       := {}             //Retorno
1109 Local  lRet       := .T.            //Retorno
1110 Local  cOrcAnt    := ""            //orcamento anterior
1111 Local  nPos       := 0              //Posição do array
1112 Local  aReg       := {}            //REGistros da tabela MBZ
1113 Local  nReg       := 0              //Registro anterior
1114 Local  cOrc       := ""            //Orcamento
1115 Local  cCup       := ""            //Cupom
1116 Local  cSerie     := ""            //Serie
1117 Local  lContinua  := .F.           //Controle de execucao
1118
1119 DbSelectArea( "MBZ" )
1120 DbSetOrder( 2 ) //MBZ_FILIAL + MBZ_SITUA + MBZ_CUPOM + MBZ_SERIE
1121 DbSeek( xFilial() + "00" )
1122
1123
1124 Do While MBZ_FILIAL + MBZ_SITUA == xFilial() + "00"
1125
1126
1127   If cOrcAnt <> (MBZ->MBZ_CUPOM + MBZ->MBZ_SERIE)
1128     STDLogCons( "STDRecEstSale" + MBZ->MBZ_NUM + STR0015 ; //" Enviando para o server Estorno da Venda do Orçamento - "
1129               +MBZ->MBZ_CUPOM + "/" + MBZ->MBZ_SERIE )
1130     cOrcAnt := MBZ->MBZ_CUPOM + MBZ->MBZ_SERIE
1131     cOrc    := MBZ->MBZ_NUM
1132     cCup    := MBZ->MBZ_CUPOM
1133     cSerie  := MBZ->MBZ_SERIE
1134   EndIf
1135
1136   aMBZ := {}
1137   aReg := {}

```

```

1138     nPos := 0
1139
1140     Do While cOrcAnt == MBZ->MBZ_CUPOM + MBZ->MBZ_SERIE
1141
1142         aAdd(aMBZ, Array( FCount() ) )
1143         nPos := nPos + 1
1144
1145
1146         For nI := 1 To FCount()
1147             aMBZ[nPos, nI] := { FieldName( nI ), FieldGet( nI ) }
1148         Next
1149
1150         aAdd(aReg, Recno())
1151
1152         MBZ->(DbSkip(1))
1153
1154     EndDo
1155
1156     nReg := Recno()
1157
1158     // Realizado comunicacao via componente de comunicacao
1159     // Efetua a gravacao do cancelamento no server
1160     lContinua := STBRemoteExecute( "GeraMBZ" , { "MBZ", aMBZ } , NIL, .F. , @aRet )
1161
1162     If lContinua .AND. ValType( aRet ) == "A"
1163
1164         If Left( aRet[1], 2 ) == "OK"
1165
1166             // Se recebeu retorno OK executa confirmacao
1167             lContinua := STBRemoteExecute( "ConfMBZ" , { aRet[2] } , NIL, .F. , @lRet )
1168
1169             STDLogCons(cAlias + STR0016) //" gravado no server."
1170
1171             If lContinua .AND. lRet
1172                 aMBZ := { { "MBZ_SITUA", "TX" } } // "TX" - Foi Enviado ao Server
1173                 For nI := 1 to Len(aReg)
1174                     MBZ->(DbGoTo(aReg[nI]))
1175                     STFSaveTab( "MBZ", aMBZ, .F. )
1176                     STDLogCons(cAlias + STR0017 + cAlias+"_SITUA" + ": " + "OK" + " Recno: " + cValToChar(aRecnos[nI])) //" Atualizado. " ### " Recno: "
1177                 Next
1178             EndIf
1179         Else
1180
1181             aMBZ := { { "MBZ_SITUA", Left( aRet[1], 2 ) } } // "Erro" - Chave duplicada
1182             For nI := 1 to Len(aReg)
1183                 MBZ->(DbGoTo(aReg[nI]))
1184                 STFSaveTab( "MBZ", aMBZ, .F. )
1185                 STDLogCons(cAlias + STR0018 + cAlias+"_SITUA" + ": " + Left( aRet[1], 2 ) + " Recno: " + cValToChar(aRecnos[nI])) //" Não Atualizado. " ### " Recno: "
1186             Next
1187             EndIf
1188         Else
1189

```

```

1190      STDLogCons(STR0014) //"Erro de comunicacao com servidor"
1191      Exit
1192  EndIf
1193
1194
1195      DbGoTo(nReg)
1196
1197 EndDo
1198
1199 RestArea(aArea)
1200
1201 Return Nil
1202
1203
1204 //-----
1205 /*{Protheus.doc} STDRecTabServer
1206 Grava Alias no server
1207
1208 @Param  cAlias      - Alias que sera usado
1209 @Param  nOrder      - Codigo de ordenacao
1210 @Param  cChave      - Campos de busca
1211 @Param  cBusca      - Conteudo de busca
1212 @Param  cConfLocal  - String de confirmacao de gravacao do campo Local
1213 @Param  cConfServer - String de confirmacao de gravacao do campo no server
1214 @Param  cEstacaoIni - Estacao
1215 @Param  cFunc       - Funcao a ser executado no server
1216 @param  lDefault    - ??
1217 @param  lPdvOnCPdv  - lógico - estação pdv online com central de pdv
1218
1219 @return Nil
1220 @obs
1221 @sample
1222 */
1223 //-----
1224 Function STDRecTabServer( cAlias      , nOrder      , cChave      , cBusca      , ;
1225                          cConfLocal , cConfServer , cEstacaoIni , cFunc      , ;
1226                          lDefault   , lPdvOnCPdv)
1227
1228 Local aRegnos      := {}
1229 Local nX           := 0           // Contador
1230 Local lRetServer   := .F.         // Retorno do Servidor
1231 Local cNameField   := ""          // Nome do campo situa para o Alias atual
1232 Local lRet         := .F.         // Retorno
1233 Local xRet         := Nil         // Retorno
1234 Local aData        := {}         // Array com dados a subir para o server
1235 Local lContinua    := .F.         // Controle de execucao
1236 Local nRecServer   := 0           // numero do Recno a ser confirmado no servidor
1237 Local aArea        := GetArea()// Guarda area atual
1238 Local nCont        := 0
1239 Local cRetSrv      := ""
1240 Local cLogICpo     := ""
1241 Local cLogACpo     := ""

```

```

1242 Local lDecode      := .T.
1243 Local cXMLEPaf      := ""
1244 Local cXMLRPaf      := ""
1245 Local cCurrTab      := ""
1246 Local cFieldName    := ""
1247
1248 Default      cAlias      := ""
1249 Default      nOrder      := 0
1250 Default      cChave      := ""
1251 Default      cBusca      := ""
1252 Default      cConfLocal  := ""
1253 Default      cConfServer := ""
1254 Default      cEstacaoIni := ""
1255 Default      cFunc       := ""
1256 Default      lPdvOnCPdv  := .F.
1257
1258 // Formata campo NOMETABELA_SITUA
1259 // Exemplo: Iniciando com "S": SL1->L1_SITUA, caso contrário MDZ->MDZ_SITUA
1260 If Substr(Upper(cAlias), 1, 1) == "S"
1261     cCurrTab      := AllTrim(Upper(Substr(cAlias, 2, 2)))
1262     cNameField    := cCurrTab + "_SITUA"
1263
1264     //Tratamento para os campos: _USERLGI, _USERLGA
1265     cLogICpo      := cCurrTab + "_USERLGI"
1266     cLogACpo      := cCurrTab + "_USERLGA"
1267
1268     //XML do PAF
1269     cXMLEPaf      := cCurrTab + "_XMLEPAF"
1270     cXMLRPaf      := cCurrTab + "_XMLRPAF"
1271 Else
1272     cCurrTab      := AllTrim(Upper(cAlias))
1273     cNameField    := cCurrTab + "_SITUA"
1274
1275     //Tratamento para os campos: _USERGI, _USERGA
1276     cLogICpo      := cCurrTab + "_USERGI"
1277     cLogACpo      := cCurrTab + "_USERGA"
1278
1279     //XML do PAF
1280     cXMLEPaf      := cCurrTab + "_XMLEPA"
1281     cXMLRPaf      := cCurrTab + "_XMLRPA"
1282 EndIf
1283
1284 DbSelectArea(cAlias)
1285 DbSetOrder(nOrder)
1286
1287 If DbSeek(cBusca)
1288
1289     nCampos := FCount()
1290
1291     While (cAlias)->(!Eof()) .AND. &(cChave) == cBusca .AND. nCont <= 300
1292
1293         nCont      := nCont + 1

```

```

1294     cRetSrv := ""
1295
1296     For nX := 1 To nCampos
1297         cFieldName := AllTrim( FieldName( nX ) )
1298
1299         //Tratamento para os campos: _USERLGI, _USERLGA ou _USERGI, _USERGA
1300         If cLogICpo == cFieldName .Or. cLogACpo == cFieldName .Or.;
1301             cXMLRPaf == cFieldName .Or. cXMLEPaf == cFieldName
1302
1303             AAdd( aData , { cFieldName, Encode64( FieldGet(nX)) })
1304         Else
1305             AAdd( aData , { cFieldName, FieldGet( nX ) } )
1306         EndIf
1307     Next i
1308
1309     If Len(aData) > 0
1310
1311         LjGrvLog( "STDRecTabServer" , "Subir tabela: " + cAlias)
1312         Conout("STDRecTabServer - Subir tabela: " + cAlias)
1313
1314         // Realizado comunicacao via componente de comunicacao
1315         // Efetua a gravacao dos dados no server de acordo com o ALIAS
1316         Do Case
1317             Case !Empty(cFunc) // Rotina especifica
1318                 lContinua := STBRemoteExecute( cFunc , { aData } , NIL, .F. , @xRet )
1319             Case cAlias == "SFI" // Resumo Reducao Z
1320                 lContinua := STBRemoteExecute( "GeraSFI" , { aData, lDecode } , NIL, .F. , @xRet )
1321             Case cAlias == "SE5" // Movimentacao Bancaria
1322                 lContinua := STBRemoteExecute( "GeraE5" , { aData, lDecode } , NIL, .F. , @xRet,/*6*/,/*7*/,/*8*/,/*9*/,/*10*/,lPdvOnCPdv )
1323             Case cAlias == "SLX" // Movimentacao Bancaria
1324                 lContinua := STBRemoteExecute( "GeraSLX" , { aData, lDecode } , NIL, .F. , @xRet)
1325             Case cAlias == "SLW" // SLW = Movimento Processos de Venda
1326                 If Empty(DtoS(aData[aScan(aData,{|x| AllTrim(x[1]) == "LW_DTFECHA"})][2]))
1327                     // Faz abertura no server
1328                     lContinua := STBRemoteExecute( "FRT020ABR" , { aData,.T., lDecode } , NIL, .F. , @xRet )
1329                 Else
1330                     // Faz fechamento no server
1331                     lContinua := STBRemoteExecute( "FRT020FCH" , { aData,.T., lDecode } , NIL, .F. , @xRet )
1332                 EndIf
1333             Case cAlias $ "MDZ|SLT" // MDZ = Movimento por ECF ||| SLT = Conferencia de Caixa
1334                 lContinua := STBRemoteExecute( "GeraSZ" , { cAlias , aData, lDecode } , NIL, .F. , @xRet )
1335             Otherwise // Demais tabelas, rotina generica
1336                 lContinua := STBRemoteExecute( "STDRecXData" , { cAlias , aData, Nil , Nil , Nil, lDecode } , NIL, .F. , @xRet )
1337         EndCase
1338
1339         //Trata o retorno xRet
1340         If lContinua .AND. ValType(xRet) == "A"
1341             lRetServer := xRet[1] == "OK"
1342             nRecServer := xRet[2]
1343             cRetSrv := xRet[1]
1344         ElseIf lContinua .AND. ValType(xRet) == "L"

```

```

1346         lRetServer := xRet
1347     ElseIf lContinua
1348         cRetSrv := "ER"
1349     EndIf
1350
1351     If lContinua .AND. lRetServer
1352         /*
1353         Quando PAF-ECF a MDZ retorna -1 para determinados registros que não podem ser gravados
1354         duas vezes porém como é validado se nRecServer > 0, não é necessário tratar separadamente porem
1355         se feito alguma alteração tratar a MDZ como é feito no FRTA020
1356         */
1357         // Se precisa fazer confirmacao no servidor confirma de acordo com recno
1358         If nRecServer > 0
1359             lContinua := STBRemoteExecute( "STDConfRec" , { cAlias , nRecServer , cNameField , cConfServer } , NIL, .F. , @lRet )
1360         EndIf
1361
1362         If lContinua
1363             STDLogCons(cAlias + STR0016) //" gravado no server."
1364
1365             If RecLock( cAlias, .F. )
1366                 REPLACE &((cAlias)->(cNameField)) WITH cConfLocal
1367                 &(cAlias)->(dbCommit())
1368                 &(cAlias)->(MsUnlock())
1369                 STDLogCons(cAlias + STR0017 + cNameField + ": " + cConfLocal + " Recno: " + cValToChar(&(cAlias)->(Recno())) //" atualizado. -> Recno: "
1370             EndIf
1371         EndIf
1372
1373     ElseIf lContinua
1374
1375         If RecLock( cAlias, .F. )
1376             REPLACE &((cAlias)->(cNameField)) WITH cRetSrv
1377             &(cAlias)->(dbCommit())
1378             &(cAlias)->(MsUnlock())
1379
1380         EndIf
1381     Else
1382         STDLogCons(STR0014 ) //"Erro de comunicacao com servidor"
1383     EndIf
1384 EndIf
1385
1386 aData := {}
1387 lRetServer := .F.
1388 nRecServer := 0
1389
1390 /*
1391 Por garantia posiciona no registro anterior para depois
1392 ir para o primeiro, pois ao alterar um campo do indice
1393 os registros serao reordenados.
1394 */
1395
1396 If lContinua
1397     &(cAlias)->(DbSkip(-1))

```

```

1398      &(cAlias)->(DbGoTop())
1399      Else
1400      &(cAlias)->(DbSkip())
1401      EndIf
1402      EndDo
1403 EndIf
1404
1405 RestArea(aArea)
1406
1407 Return Nil
1408
1409
1410 //-----
1411 /*{Protheus.doc} STDRecOpenClose
1412 Grava Alias no server
1413
1414 @Param   cAlias      - Alias que sera usado
1415 @Param   nOrder      - Codigo de ordenacao
1416 @Param   cChave      - Campos de busca
1417 @Param   cBusca      - Conteudo de busca
1418 @Param   cConfLocal  - String de confirmacao de gravacao do campo Local
1419 @Param   cConfServer  - String de confirmacao de gravacao do campo no server
1420 @Param   cEstacaoIni - Estacao
1421
1422 @return  Nil
1423 @obs
1424 @sample
1425 */
1426 //-----
1427 Function STDRecOpenClose( cAlias      , nOrder      , cChave      , cBusca      ,
1428                          cConfLocal  , cConfServer  , cEstacaoIni ,
1429                          )
1430 Local aArea      := GetArea() // Armazena alias corrente
1431 Local nX         := 0        // Contador
1432 Local lRetServer := .F.      // Retorno do Servidor
1433 Local cNameField := ""       // Nome do campo situa para o Alias atual
1434 Local lRet       := .F.      // Retorno
1435 Local xRet       := Nil      // Retorno
1436 Local aData      := {}       // Array de dados a serem gravados no server
1437 Local lContinua  := .F.      // Controle de execucao
1438 Local cLogICpo   := ""
1439 Local cLogACpo   := ""
1440 Local lDecode    := .T.
1441
1442 Default cAlias      := ""
1443 Default nOrder      := 0
1444 Default cChave      := ""
1445 Default cBusca      := ""
1446 Default cConfLocal  := ""
1447 Default cConfServer := ""
1448 Default cEstacaoIni := ""
1449

```

```

1450 // Formata campo NOMETABELA_SITUA
1451 // Exemplo: Iniciando com "S": SL1->L1_SITUA, caso contrário MDZ->MDZ_SITUA
1452 If Substr(Upper(cAlias), 1, 1) == "S"
1453     cNameField := Substr(cAlias, 2, 2) + "_SITUA"
1454
1455     //Tratamento para os campos: _USERLGI, _USERLGA
1456     cLogICpo := AllTrim(UPPER(Substr(cAlias, 2, 2)) + "_USERLGI")
1457     cLogACpo := AllTrim(UPPER(Substr(cAlias, 2, 2)) + "_USERLGA")
1458 Else
1459     cNameField := cAlias + "_SITUA"
1460
1461     //Tratamento para os campos: _USERGI, _USERGA
1462     cLogICpo := AllTrim(UPPER(cAlias) + "_USERGI")
1463     cLogACpo := AllTrim(UPPER(cAlias) + "_USERGA")
1464 EndIf
1465
1466 //cAlias == "SLW" // SLW = Movimento Processos de Venda
1467 DbSelectArea(cAlias)
1468 DbSetOrder(nOrder)
1469 If DbSeek(cBusca)
1470
1471     nCampos := FCount()
1472
1473     While &(cChave) == cBusca .AND. !EOF()
1474
1475         For nX := 1 To nCampos
1476             //Tratamento para os campos: _USERLGI, _USERLGA ou _USERGI, _USERGA
1477             If cLogICpo == FieldName( nX ) .Or. cLogACpo == FieldName( nX )
1478                 AAdd( aData , { FieldName( nX ), Encode64( FieldGet(nX)) } )
1479             Else
1480                 AAdd( aData , { FieldName( nX ), FieldGet( nX ) } )
1481             EndIf
1482         Next i
1483
1484         If Len(aData) > 0
1485
1486             // Realizado comunicacao via componente de comunicacao
1487             // Efetua a gravacao dos dados no server de acordo com o ALIAS
1488
1489             If Empty(DtoS(aData[aScan(aData,{|x| AllTrim(x[1]) == "LW_DTFECHA"}])[2]))
1490                 // Faz abertura no server
1491                 lContinua := STBRemoteExecute( "FRT020ABR" , { aData,.T., lDecode } , NIL, .F. , @xRet )
1492             Else
1493                 // Faz fechamento no server
1494                 lContinua := STBRemoteExecute( "FRT020FCH" , { aData,.T., lDecode } , NIL, .F. , @xRet )
1495             EndIf
1496
1497             //Trata o retorno xRet
1498             If lContinua .AND. ValType(xRet) == "L"
1499                 lRetServer := xRet
1500             EndIf
1501

```

```

1502         If lContinua .AND. lRetServer
1503
1504             STDLogCons(cAlias + STR0016) //" gravado no server."
1505
1506             //Atualiza campo XX_SITUA como enviado para o server. EX: "TX" "OK"
1507             If STFSaveTab(cAlias, {{cNameField, cConfLocal}})
1508                 STDLogCons(cAlias + STR0017+ cNameField + ": " + cConfLocal + " Recno: " + cValToChar(&(cAlias)->(Recno())) //" atualizado. -> Recno: "
1509             EndIf
1510
1511         Else
1512             STDLogCons(STR0014 ) //"Erro de comunicacao com servidor"
1513         EndIf
1514
1515     EndIf
1516
1517     aData := {}
1518     lRetServer := .F.
1519     &(cAlias)->(DbSkip())
1520
1521 EndDo
1522
1523 EndIf
1524
1525 RestArea(aArea)
1526
1527 Return Nil
1528
1529 //-----
1530 /*{Protheus.doc} STDRecGenTabServer
1531 Grava Alias generico no server
1532
1533 @Param
1534 @return Nil
1535 @obs
1536 @sample
1537 */
1538 //-----
1539 Function STDRecGenTabServer()
1540
1541 Local aArea          := GetArea()           // Armazena alias corrente
1542 Local nX             := 0                   // Contador
1543 Local lRetServer      := .F.                 // Retorno do Servidor
1544 Local cNameField      := ""                 // Nome do campo situa para o Alias atual
1545 Local lRet            := .F.                 // Retorno
1546 Local xRet           := Nil                 // Retorno
1547 Local aData           := {}                 // Array com dados a subir para o server
1548 Local lContinua      := .F.                 // Controle de execucao
1549 Local nRecServer      := 0                   // numero do Recno a ser confirmado no servidor
1550 Local cAlias          := ""                 // Alias que ira subir ao server
1551 Local cFunc           := ""                 // Funcao a ser executada no server
1552 Local cConfLocal      := "TX"               // Texto para Confirmacao local
1553 Local cConfServer     := "RX"               // Texto para Confirmacao no server

```

```

1554 Local cStatusError := "ER"      // Texto para Confirmacao no server
1555 Local nCont         := 0
1556 Local lExistCli     := .F. //Cliente ja existe?
1557 Local nIndice       := 0
1558 Local cChave        := ""
1559 Local aFieldSA1 := {"A1_FILIAL", "A1_COD", "A1_LOJA", "A1_NOME", "A1_NREDUZ", "A1_CGC", "A1_END", "A1_EST", "A1_MUN", "A1_TIPO", "A1_PESSOA"} //Campos atualizados no cadastro
de clientes
1560 Local lSTDEdtSA1 := ExistBlock("STDEdtSA1") //Ponto de entrada para edição do Cadastro de Clientes
1561 Local aSTDEdtSA1 := {} //Resultado do P.E para edição de clientes
1562 Local cFieldName := "" //Nome do campo
1563 Local uValue     := NIL //Valor do campo
1564 Local cLogICpo   := "" // Nome do campo XX_USERLGI
1565 Local cLogACpo   := "" // Nome do campo XX_USERLGA
1566 Local nPosLGI    := 0 //Posicao do campo do campo XX_USERLGI
1567 Local nPosLGA    := 0 //Posicao do campo do campo XX_USERLGA
1568
1569 CoNout('Entrou na funcao - STDRecGenTabServer')
1570
1571 DbSelectArea("SLI")
1572 DbSetOrder(2) //LI_FILIAL+LI_TIPO
1573 If DbSeek(xFilial("SLI") + "UP")
1574
1575     While SLI->(!EOF()) .AND. AllTrim(SLI->LI_TIPO) == "UP" .AND. nCont <= 20
1576         lExistCli := .F.
1577         nIndice   := 0
1578         cChave    := ""
1579         nCont := nCont + 1
1580
1581         If !Empty(SLI->LI_ALIAS) .AND. SLI->LI_UPREC > 0
1582             CoNout('STDRecGenTabServer - Encontrou ' + SLI->LI_ALIAS + ' Recno: ' + AllTrim(Str(SLI->LI_UPREC)))
1583
1584             cAlias := AllTrim(SLI->LI_ALIAS)
1585             // Formata campo NOMETABELA_SITUA
1586
1587             If Substr(Upper(cAlias), 1, 1) == "S"
1588                 cNameField := Substr(cAlias, 2, 2) + "_SITUA"
1589             Else
1590                 cNameField := cAlias + "_SITUA"
1591             EndIf
1592
1593             If !Empty(SLI->LI_FUNC)
1594                 cFunc := AllTrim(SLI->LI_FUNC)
1595             EndIf
1596
1597             // Posiciona no registro a subir para o server
1598             DbSelectArea(cAlias)
1599             &(cAlias)->(DbGoto(SLI->LI_UPREC))
1600
1601             If &(cAlias)->(&(cNameField)) == "00"
1602
1603                 If AllTrim(cAlias) == 'SA1'
1604                     lContinua := STBRemoteExecute( "STDPesqCli" , { SA1->A1_FILIAL, SA1->A1_COD, SA1->A1_LOJA } , NIL, .F. , @lExistCli )

```

```

1605         If lContinua
1606             nIndice             := 1
1607             cChave              := SA1->A1_FILIAL + SA1->A1_COD + SA1->A1_LOJA
1608
1609             If lSTDEdtSA1
1610                 ConOut("STDRecGenTabServer - Executando o p.E. STDEdtSA1")
1611                 aSTDEdtSA1 := ExecBlock("STDEdtSA1",.F.,.F.,{lExistCli})
1612                 ConOut("STDRecGenTabServer - Executado o p.E. STDEdtSA1 - campos editado" + cValToChar(Len(aSTDEdtSA1)))
1613             Else
1614                 If lExistCli
1615                     //Edição, carrega somente os campos que podem ser alterados
1616                     aSTDEdtSA1 := aClone(aFieldSA1)
1617                     ConOut("STDRecGenTabServer - Padrão campos alterados" + cValToChar(Len(aSTDEdtSA1)))
1618
1619                 Else
1620                     //Se for inclusão, atualiza o SA1 inteiro
1621                     For nX := 1 To FCount()
1622                         AAdd( aSTDEdtSA1 , FieldName( nX ) )
1623                     Next nX
1624                     ConOut("STDRecGenTabServer - Padrão campos Inseridos" + cValToChar(Len(aSTDEdtSA1)))
1625                 EndIf
1626             EndIf
1627
1628             For nX := 1 to Len(aSTDEdtSA1)
1629                 cFieldName := aSTDEdtSA1[nX]
1630                 uValue      := FieldGet(FieldPos(cFieldName)) //Valor do campo
1631                 AAdd( aData , {cFieldName , uValue } )
1632             Next nX
1633
1634         EndIf
1635     Else
1636         CoNout('STDRecGenTabServer - Encontrou o alias ' + cAlias + ', Recno: ' + AllTrim(Str(SLI->LI_UPREC)))
1637         For nX := 1 To FCount()
1638             cFieldName := FieldName( nX )
1639             uValue      := FieldGet( nX )
1640             AAdd( aData , { cFieldName, uValue } )
1641         Next i
1642     EndIf
1643
1644     ElseIf AllTrim(cAlias) == 'SA1'
1645         //Apaga o registro ja processado da SLI
1646         If RecLock("SLI",.F.)
1647             SLI->(DbDelete())
1648             SLI->(MsUnlock())
1649         EndIf
1650         CoNout('STDRecGenTabServer - Nao estava como A1_SITUA == 00, entao foi excluido da SLI')
1651     EndIf
1652
1653     CoNout('STDRecGenTabServer - Array aData retornou ' + AllTrim(Str(Len(aData))))
1654     If Len(aData) > 0
1655
1656         // Realizado comunicacao via componente de comunicacao

```

```

1657 // Efetua a gravacao dos dados no server de acordo com o ALIAS
1658 If Empty(cFunc)
1659     //Formata campo NOMETABELA_USERLG?, Exemplo: Iniciando com "S": SLT->LT_USERLGI, caso contrário MDZ->MDZ_USERGI
1660     If Substr(Upper(cAlias), 1, 1) == "S"
1661         cLogICpo := Substr(cAlias, 2, 2) + "_USERLGI"
1662         cLogACpo := Substr(cAlias, 2, 2) + "_USERLGA"
1663     Else
1664         cLogICpo := cAlias + "_USERGI"
1665         cLogACpo := cAlias + "_USERGA"
1666     EndIf
1667
1668     If (nPosLGI := AScan( aData, { |x| x[1] == cLogICpo } )) > 0
1669         aData[nPosLGI][2] := Encode64(aData[nPosLGI][2])
1670     EndIf
1671
1672     If (nPosLGA := AScan( aData, { |x| x[1] == cLogACpo } )) > 0
1673         aData[nPosLGA][2] := Encode64(aData[nPosLGA][2])
1674     EndIf
1675
1676     // Rotina generica
1677     lContinua := STBRemoteExecute( "STDRecXData" , { cAlias , aData, !lExistCli, nIndice, cChave, .T. } , NIL, .F. , @xRet )
1678 Else
1679     // Executa pela rotina especificada
1680     lContinua := STBRemoteExecute( cFunc , { aData } , NIL, .F. , @xRet )
1681 EndIf
1682
1683 //Trata o retorno xRet
1684 If lContinua .AND. ValType(xRet) == "A"
1685     lRetServer := xRet[1] == "OK"
1686     nRecServer := xRet[2]
1687     CoNout('STDRecGenTabServer - Retorno do RemoteExecute Ok')
1688 ElseIf lContinua .AND. ValType(xRet) == "L"
1689     lRetServer := xRet
1690     CoNout('STDRecGenTabServer - Retorno do xRet foi L')
1691 EndIf
1692
1693 If lContinua .AND. lRetServer
1694
1695     // Se precisa fazer confirmacao no servidor confirma de acordo com recno
1696     If nRecServer > 0
1697         lContinua := STBRemoteExecute( "STDConfRec" , { cAlias , nRecServer , cNameField , cConfServer } , NIL, .F. , @lRet )
1698     EndIf
1699
1700     If lContinua
1701
1702         STDLogCons(cAlias + STR0016) //" gravado no server."
1703
1704
1705         If RecLock( cAlias, .F. )
1706
1707             REPLACE &((cAlias)->(cNameField)) WITH cConfLocal
1708             &(cAlias)->(dbCommit())

```

```

1709             &(cAlias)->(MsUnlock())
1710             STDLogCons(cAlias + STR0017 + cNameField + ": " + cConfLocal + " Recno: " + cValToChar(&(cAlias)->(Recno())) // " atualizado. -> Recno: "
1711
1712             // Apaga o registro ja processado da SLI
1713             If RecLock("SLI",.F.)
1714                 SLI->(DbDelete())
1715                 SLI->(MsUnlock())
1716             EndIf
1717
1718             Else
1719                 CoNout('STDRecGenTabServer - Retorno da confirmacao (STDConfRec) .F.')
1720             EndIf
1721
1722             EndIf
1723
1724             ElseIf lContinua .AND. !lRetServer
1725
1726                 STDLogCons(cAlias + " Erro ao gravar no server. Alias: " + cAlias ) //" Erro ao gravar no server"
1727
1728                 If RecLock( cAlias, .F. )
1729
1730                     REPLACE &((cAlias)->(cNameField)) WITH cStatusError
1731                     &(cAlias)->(dbCommit())
1732                     &(cAlias)->(MsUnlock())
1733
1734                     // Apaga o registro ja processado da SLI
1735                     If RecLock("SLI",.F.)
1736                         SLI->(DbDelete())
1737                         SLI->(MsUnlock())
1738                     EndIf
1739
1740                 EndIf
1741
1742                 Else
1743                     STDLogCons(STR0014 ) //"Erro de comunicacao com servidor"
1744                 EndIf
1745
1746             EndIf
1747
1748             aData := {}
1749             lRetServer := .F.
1750             nRecServer := 0
1751             cFunc := ""
1752             cAlias := ""
1753
1754             EndIf
1755
1756             SLI->(DbSkip())
1757
1758         End
1759
1760 EndIf

```

```

1761
1762 RestArea(aArea)
1763
1764 Return Nil
1765
1766
1767 //-----
1768 /*{Protheus.doc} STDConfRec
1769 Confirma a Gravacao dos dados recebidos
1770
1771 @Param cAlias      - Alias
1772 @Param nRecno      - Recno
1773 @Param cNameField  - Nome do campo a ser atualizado
1774 @Param cTextConf   - Texto de confirmacao a ser gravado no campo
1775
1776 @return lRet       Retona se executou corretamente a funcao
1777 @obs
1778 @sample
1779 */
1780 //-----
1781 Function STDConfRec( cAlias , nRecno , cNameField , cTextConf )
1782
1783 Local lRet := .F.      // Retorno
1784
1785 Default cAlias      := ""
1786 Default nRecno      := 0
1787 Default cNameField  := ""
1788 Default cTextConf   := "RX"
1789
1790 ParamType 0 Var      cAlias      As Character      Default      ""
1791 ParamType 1 Var      nRecno      As Numeric        Default      0
1792 ParamType 2 Var      cNameField  As Character      Default      ""
1793 ParamType 3 Var      cTextConf   As Character      Default      "RX"
1794
1795
1796 &(cAlias)->(DbGoto(nRecno))
1797 lRet := STFSaveTab(cAlias, {{cNameField, cTextConf}})      // "RX" "OK" - Foi Recebido Pelo Server
1798
1799 Return lRet
1800
1801
1802 //-----
1803 /*{Protheus.doc} STDXRzArray
1804 Gera um array do alias recebido e que esta com _SITUA = "00"
1805
1806 @Param cAlias      - Alias que sera usado
1807 @Param nOrder      - Codigo de ordenacao
1808 @Param cChave      - Campos de busca
1809 @Param cBusca      - Conteudo de busca
1810 @Param aRecnos     - Recno dos registros selecionados
1811
1812 @return aRet       array com dados do Alias

```

```

1813 @obs
1814 @sample
1815 */
1816 //-----
1817
1818
1819 Function STDXRzArray(  cAlias , nOrder, cChave, cBusca ,;
1820                       aRecnos
1821 )
1822 Local   aArea      := GetArea()      // Armazena alias corrente
1823 Local nCampos      := 0              // Quantidade de campos do filtro atual
1824 Local aRet         := {}            // Array do Alias
1825 Local i           := 0              // Contador
1826
1827 Default cAlias      := ""           // Alias que sera usado
1828 Default nOrder      := 0           // Codigo de ordenacao
1829 Default cChave      := ""           // Campos de busca
1830 Default cBusca      := ""           // Conteudo de busca
1831 Default aRecnos     := {}           // Recno dos registros selecionados. Chamar por referencia
1832
1833 ParamType 0 Var      cAlias      As Character   Default   ""
1834 ParamType 1 Var      nOrder      As Numeric    Default   0
1835 ParamType 2 Var      cChave      As Character   Default   ""
1836 ParamType 3 Var      cBusca      As Character   Default   ""
1837 ParamType 4 Var      aRecnos     As Array       Default   {}
1838
1839
1840 DbSelectArea(cAlias)
1841 DbSetOrder(nOrder)
1842 If DbSeek(cBusca)
1843
1844     nCampos := FCount()
1845
1846     // Monta a primeira posicao do array com o nome dos campos
1847     aRet := { Array(nCampos) }
1848     For i := 1 To nCampos
1849         aRet[1][i] := FieldName(i)
1850     Next i
1851
1852     While &(cChave) == cBusca .AND. !EOF()
1853
1854         AAdd(aRet, Array(nCampos))
1855
1856         For i := 1 To nCampos
1857
1858             // Monta as demais posicoes do array com o conteudo
1859             aRet[Len(aRet)][i] := FieldGet(i)
1860
1861         Next i
1862
1863         //Adiciona Recnos para atualizacao do campo XX_SITUA
1864         AAdd(aRecnos, &(cAlias)->(Recno()) )

```

```

1865
1866      &(cAlias)->(DbSkip())
1867
1868      EndDo
1869
1870 EndIf
1871
1872 RestArea(aArea)
1873
1874 Return aRet
1875
1876
1877 //-----
1878 /*{Protheus.doc} STDArray
1879 Gera um array do alias recebido e que esta com _SITUA = "00"
1880
1881 @Param cAlias      - Alias que sera usado
1882 @Param nOrder      - Codigo de ordenacao
1883 @Param cChave      - Campos de busca
1884 @Param cBusca      - Conteudo de busca
1885 @Param aRecnos     - Recno dos registros selecionados
1886
1887 @return aRet      array com dados do Alias
1888 @obs
1889 @sample
1890 */
1891 //-----
1892
1893
1894 Function STDArray(      cAlias , nOrder, cChave, cBusca ,;
1895                      aRecnos , cFunction
1896                      )
1897 Local   aArea      := GetArea()    // Armazena alias corrente
1898 Local nCampos      := 0            // Quantidade de campos do filtro atual
1899 Local aRet         := {}          // Array do Alias
1900 Local i            := 0            // Contador
1901
1902 Default cAlias      := ""          // Alias que sera usado
1903 Default nOrder      := 0           // Codigo de ordenacao
1904 Default cChave      := ""          // Campos de busca
1905 Default cBusca      := ""          // Conteudo de busca
1906 Default aRecnos     := {}          // Recno dos registros selecionados. Chamar por referencia
1907 Default cFunction   := ""          // Function
1908
1909 ParamType 0 Var      cAlias      As Character   Default   ""
1910 ParamType 1 Var      nOrder      As Numeric    Default   0
1911 ParamType 2 Var      cChave      As Character   Default   ""
1912 ParamType 3 Var      cBusca      As Character   Default   ""
1913 ParamType 4 Var      aRecnos     As Array       Default   {}
1914 ParamType 5 Var      cFunction   As Character   Default   ""
1915
1916 DbSelectArea(cAlias)

```

```

1917 DbSetOrder(nOrder)
1918 If DbSeek(cBusca)
1919
1920     nCampos := FCount()
1921
1922     While &(cChave) == cBusca .AND. !EOF()
1923
1924         AAdd(aRet, Array(nCampos))
1925
1926         For i := 1 To nCampos
1927
1928             // Monta as demais posicoes do array com o conteudo
1929             aRet[Len(aRet)][i] := { FieldName( i ), FieldGet( i ) }
1930         Next i
1931
1932         //Adiciona Recnos para atualizacao do campo XX_SITUA
1933         Aadd(aRecnos, &(cAlias)->(Recno()) )
1934
1935         &(cAlias)->(DbSkip())
1936
1937     EndDo
1938
1939 EndIf
1940
1941 RestArea(aArea)
1942
1943 Return aRet
1944
1945
1946 //-----
1947 /*{Protheus.doc} STDRecXRzData
1948 Grava dados da Reducao Z
1949 @Param cAlias      - Alias que sera usado
1950 @Param aData       - Array com dados da reducao
1951 @return lRet       Se executou corretamente
1952 @obs
1953 @sample
1954 */
1955 //-----
1956 Function STDRecXRzData( cAlias , aData )
1957
1958 Local   aArea           := GetArea()      // Armazena alias corrente
1959 Local nI                := 0              // Contador
1960 Local nJ                := 0              // Contador
1961 Local aRec              := {}             // Array para fazer garvacao dos dados
1962 Local cNameField        := ""             // Nome do campo situa para o Alias atual
1963 Local lRet              := .F.            // Retorno
1964
1965 Default cAlias := ""
1966 Default aData := {}
1967
1968 ParamType 0 Var      cAlias      As Character      Default      ""

```

```

1969 ParamType 1 Var  aData      As Array    Default      {}
1970
1971 // Formata campo NOMETABELA_SITUA, Exemplo:
1972 // Iniciando com "S": L1_SITUA, caso contrario MDZ_SITUA
1973 If Substr(Upper(cAlias), 1, 1) == "S"
1974     cNameField := Substr(cAlias, 2, 2) + "_SITUA"
1975 Else
1976     cNameField := cAlias + "_SITUA"
1977 EndIf
1978
1979 // Gera dados
1980 For nI := 2 To Len( aData )
1981
1982     aRec := Array(Len(aData[1]))
1983
1984     // Transforma o array recebido para o formato: [NOME]; [VALOR]
1985     // Pois e recebido em outro formato para diminuir o tamanho do arquivo
1986     For nJ := 1 To Len(aData[1])
1987         aRec[nJ] := {aData[1][nJ], aData[nI][nJ]}
1988     Next nJ
1989
1990     // "TX" - Foi Transferido Para o Server
1991     aRec[AScan(aRec, {|x|x[1]== cNameField })][2] := "TX"
1992
1993     lRet := STFSaveTab( cAlias , aRec, .T.)
1994
1995 Next nI
1996
1997 RestArea(aArea)
1998
1999 Return lRet
2000
2001
2002 //-----
2003 /*{Protheus.doc} STDRecXData
2004 Grava dados nas tabelas genéricas
2005 @Param cAlias      - Alias que sera usado
2006 @Param aData       - Array com dados dos campos
2007 @Param lAppend     - Define se vai inserir novo ou alterar registro existente
2008 @Param nIndice     - Indice da tabela a ser utilizado
2009 @Param cChave      - chave a ser utilizada no seek
2010 @Param lDecode     - Formata campo NOMETABELA_USERLG
2011 @return lRet      Se executou corretamente
2012 @obs
2013 @sample
2014 */
2015 //-----
2016 Function STDRecXData( cAlias , aData, lAppend , nIndice , cChave, lDecode)
2017
2018 Local lRet          := .F.      // Retorno
2019 Local aArea         := GetArea()
2020 Local nPosLGI       := 0        // Log de inclusao de usuario

```

```

2021 Local nPosLGA      := 0          // Log de alteração de usuario
2022 Local cLogICpo     := ""
2023 Local cLogACpo     := ""
2024
2025 Default cAlias      := ""
2026 Default aData       := {}
2027 Default lAppend     := .T. //Padrão, insere o registro
2028 Default nIndice     := 0
2029 Default cChave      := ""
2030 Default lDecode     := .F.
2031
2032 ParamType 0 Var      cAlias      As Character   Default   ""
2033 ParamType 1 Var      aData       As Array      Default   {}
2034 ParamType 2 Var      lAppend     As Logical    Default   .T.
2035 ParamType 3 Var      nIndice     As Numeric    Default   0
2036 ParamType 3 Var      cChave      As Character   Default   ""
2037
2038 //Se não encontrou, insere o registro
2039
2040 If !lAppend .AND. nIndice > 0
2041     (cAlias)->(DbSetOrder(nIndice))
2042     lAppend := !(cAlias)->(DbSeek(cChave))
2043 EndIf
2044
2045 If lDecode
2046     //Formata campo NOMETABELA_USERLG?, Exemplo: Iniciando com "S": SLT->LT_USERLGI, caso contrário MDZ->MDZ_USERGI
2047     If Substr(Upper(cAlias), 1, 1) == "S"
2048         cLogICpo := Substr(cAlias, 2, 2) + "_USERLGI"
2049         cLogACpo := Substr(cAlias, 2, 2) + "_USERLGA"
2050     Else
2051         cLogICpo := cAlias + "_USERGI"
2052         cLogACpo := cAlias + "_USERGA"
2053     EndIf
2054
2055     If (nPosLGI := AScan( aData, { |x| x[1] == cLogICpo } )) > 0
2056         aData[nPosLGI][2] := Decode64(aData[nPosLGI][2])
2057     EndIf
2058
2059     If (nPosLGA := AScan( aData, { |x| x[1] == cLogACpo } )) > 0
2060         aData[nPosLGA][2] := Decode64(aData[nPosLGA][2])
2061     EndIf
2062 EndIf
2063
2064
2065 //Se for atualizar, o registro deve estar posicionado
2066 lRet := STFSaveTab( cAlias , aData , lAppend)
2067
2068
2069 RestArea(aArea)
2070 Return lRet
2071
2072

```

```

2073 //-----
2074 /*
2075     Funcoes para execucao da subida da venda para central de PDV
2076     com componete de comunicao. pois o mesmo nao deixava habilitar a mesma
2077     funcao para dois perfis diferentes
2078 */
2079 //-----
2080 Function _GeraL1L2L4( aSL1, aSL2, aSL4, cEstacao )
2081 Return GeraL1L2L4( aSL1, aSL2, aSL4, cEstacao )
2082
2083 //-----
2084 Function _ConfL1L2L4(cNum, cEstacao, lGravouOk)
2085 Default lGravouOk := .F.
2086 Return ConfL1L2L4(cNum, cEstacao, lGravouOk )
2087
2088 //-----
2089 Function _STDCancRec( cRegistros, cEmp, cFil )
2090 Return STDCancRec( cRegistros, cEmp, cFil )
2091
2092 //-----
2093 Function _GeraMBZ(cAlias, aMBZREg)
2094 Return GeraMBZ(cAlias, aMBZREg)
2095
2096 //-----
2097 Function _ConfMBZ(aReg)
2098 Return ConfMBZ(aReg)
2099
2100 //-----
2101 Function _GeraSFI( aSFI )
2102 Return GeraSFI( aSFI )
2103
2104 //-----
2105 Function _GeraE5( aSE5 )
2106 Return GeraE5( aSE5 )
2107
2108 //-----
2109 Function _GeraSLX( aSLX )
2110 Return GeraSLX( aSLX )
2111
2112 //-----
2113 Function _FRT020ABR(aSLW,lFunc)
2114 Return FRT020ABR(aSLW,lFunc)
2115
2116 //-----
2117 Function _FRT020FCH(aSLW,aSLT,lFunc)
2118 Return FRT020FCH(aSLW,aSLT,lFunc)
2119
2120 //-----
2121 Function _GeraSZ(cAlias, aSZ)
2122 Return GeraSZ(cAlias, aSZ)
2123
2124 //-----

```

```

2125 Function _STDRecXData( cAlias , aData, lAppend , nIndice , cChave, lDecode )
2126 Return STDRecXData( cAlias , aData, lAppend , nIndice , cChave, lDecode)
2127
2128 //-----
2129 Function _STDConfRec( cAlias , nRecno , cNameField , cTextConf )
2130 Return STDConfRec( cAlias , nRecno , cNameField , cTextConf )
2131 //-----
2132 /*/{Protheus.doc} STDGrvCli
2133 Subida do cadastro de cliente da central para a retaguarda
2134 @Param
2135 @return
2136 @obs
2137 @sample
2138 */
2139 //-----
2140 Function STDGrvCli()
2141
2142 Local aArea      := GetArea()    // Guarda area
2143 Local aData      := {}          //Armazena os campos com seus valores
2144 Local nX         := 0           //Variavel de loop
2145 Local xRet       := ''          //Variavel de retorno do deploy
2146 Local lContinua := .F.         //Confirmacao do deploy
2147 Local lRetServer := .F.         //Retorno do server
2148 Local lExistCli  := .F.         //Cliente ja existe?
2149 Local cFieldName := ""          //Nome do Campo
2150 Local uValue     := ""          //Valor do Campo
2151 Local nPosCodCli := 0           //Posicao que vai estar o cod do cliente no array aData
2152 Local nPosCodLoj := 0           //Posicao que vai estar o cod da loja no array aData
2153
2154 CoNout('Entrou na funcao de subida do cliente -> STDGrvCli')
2155
2156 DbSelectArea('SA1')
2157 SA1->(DbSetOrder(12)) //A1_FILIAL+A1_SITUA
2158
2159 While SA1->( DbSeek(xFilial("SA1")+"00")) .AND. !EOF()
2160
2161     CoNout('Encontrou cliente para subir -> STDGrvCli')
2162
2163     Sleep(5000)
2164     For nX := 1 To SA1->(FCount())
2165         cFieldName := FieldName( nX )
2166         uValue := FieldGet( nX )
2167         If AllTrim(cFieldName) == 'A1_SITUA'
2168             uValue := ''
2169         ElseIf AllTrim(FieldName( nX )) $ 'A1_USERLGI|A1_USERLGA'
2170             uValue := EnCode64(uValue)
2171         EndIf
2172
2173         If cFieldName == 'A1_COD'
2174             nPosCodCli := nX
2175         ElseIf cFieldName == 'A1_LOJA'
2176             nPosCodLoj := nX

```

```

2177         EndIf
2178
2179         AAdd( aData , { cFieldName , uValue } )
2180     Next nX
2181
2182
2183     If Len(aData) > 0
2184         lContinua := STBRemoteExecute( "STDPesqCli" , { xFilial("SA1"), aData[nPosCodCli][2], aData[nPosCodLoj][2] } , NIL, .F. , @lExistCli )
2185     EndIf
2186
2187     CONOUT("STDPesqCli - Cliente: "+AllToChar(aData[1][2])+" - Loja: "+AllToChar(aData[2][2])+" - Retorno lExistCli: "+AllToChar(lExistCli))
2188
2189     If Len(aData) > 0 .AND. lContinua
2190
2191         lContinua := STBRemoteExecute( "STDRecXData" , { 'SA1' , aData, !lExistCli , 1 , xFilial("SA1")+aData[nPosCodCli][2]+aData[nPosCodLoj][2] , .T.} , NIL, .F. , @xRet )
2192         //Trata o retorno xRet
2193         If lContinua .AND. ValType(xRet) == "A"
2194             lRetServer := xRet[1] == "OK"
2195             nRecServer := xRet[2]
2196         ElseIf lContinua .AND. ValType(xRet) == "L"
2197             lRetServer := xRet
2198         EndIf
2199
2200         If lContinua .AND. lRetServer
2201             RecLock('SA1',.F.)
2202             SA1->A1_SITUA := 'TX'
2203             MsUnlock()
2204         ElseIf lContinua .AND. !lRetServer
2205             RecLock('SA1',.F.)
2206             SA1->A1_SITUA := 'ER'
2207             MsUnlock()
2208         EndIf
2209     EndIf
2210
2211     nX := 1
2212     aData := {}
2213     SA1->(DbSkip())
2214
2215 EndDo
2216
2217 RestArea(aArea)
2218
2219 Return .T.
2220
2221 //-----
2222 /*{Protheus.doc} STDPesqCli
2223 Verifica se o cliente na existe na base da retaguarda
2224 @Param      cFil - Filial do cliente
2225 @Param      cCodCli -Codigo do cliente
2226 @Param      cCodLoja -Codigo da loja
2227 @return
2228 @obs

```

```

2229 @sample
2230 /*/
2231 //-----
2232 Function STDPesqCli(cFil, cCodCli, cCodLoja)
2233
2234 Local lRet := .F.
2235
2236 DbSelectArea("SA1")
2237 SA1->(DbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
2238
2239 cFil := PadR(cFil,TamSX3('A1_FILIAL')[1])
2240 cCodCli := PadR(cCodCli,TamSX3('A1_COD')[1])
2241 cCodLoja := PadR(cCodLoja,TamSX3('A1_LOJA')[1])
2242
2243 If SA1->(DbSeek(cFil + cCodCli + cCodLoja))
2244     lRet := .T.
2245 Else
2246     lRet := .F.
2247 EndIf
2248
2249 Return lRet
2250
2251
2252 //-----
2253 /*/{Protheus.doc} STDConGrvB
2254 Verifica se foi executado LjGrvBatch na base da retaguarda
2255 consultando se L1_SITUA = "OK"
2256 Executo somente de Central de PDV e Ambiente Central
2257
2258 @Param      cFil - Filial do cliente
2259 @Param      cCodCli -Codigo do cliente
2260 @Param      cCodLoja -Codigo da loja
2261 @return
2262 @obs
2263 @sample
2264 /*/
2265 //-----
2266 Function STDConGrvB( cPDV, cDoc, lDocNf, cSerPed, cDocPed, cSerie )
2267 Local aAreaSL1      := SL1->(GetArea()) // Guarda area
2268 Local lRet          := .F.             // Variável lógica
2269 Local aRet          := {}              // Retorno
2270 Local lSeek         := .F. //Armazena um logico para informar se encontrou a venda ou nao
2271 Local cAuxSeek      := ""
2272
2273 Default lDocNf := .F.
2274 Default cSerPed := ""
2275 Default cDocPed := ""
2276 Default cSerie := ""
2277
2278 DbSelectArea("SL1")
2279
2280 If !lDocNf

```

```

2281   if Empty(cSerie)
2282       SL1->( DbSetOrder(8) )           //L1_FILIAL+L1_PDV+L1_DOC
2283       cAuxSeek := xFilial("SL1") + cPDV + cDoc
2284   else
2285       SL1->( DbSetOrder(2) )           //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
2286       cAuxSeek := xFilial("SL1") + cSerie + cDoc + cPDV
2287   endif
2288
2289   If SL1->(DbSeek( cAuxSeek ))
2290       LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_PDV/L1_DOC:",SL1->L1_PDV+"/"+SL1->L1_SERIE+"/"+SL1->L1_DOC)
2291       LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_SITUA:",SL1->L1_SITUA)
2292       lSeek := .T.
2293   Else
2294       LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_PDV/L1_DOC não encontrada:",cPdv+"/"+cDoc)
2295   EndIf
2296
2297 Else
2298
2299     SL1->(DbSetOrder(11)) //L1_FILIAL+L1_SERPED+L1_DOCPED
2300
2301     If SL1->(DbSeek( xFilial("SL1")+cSerPed+cDocPed )) .AND. AllTrim(SL1->L1_PDV) == AllTrim(cPDV)
2302         LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_SERPED/L1_DOCPED:",cSerPed+"/"+cDocPed)
2303         LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_SITUA:",SL1->L1_SITUA)
2304         lSeek := .T.
2305     Else
2306         LjGrvLog("SUBIDA_CANCELAMENTO_RET" ,"SL1->L1_SERPED/L1_DOCPED não encontrada:",cSerPed+"/"+cDocPed)
2307     EndIf
2308
2309 EndIf
2310
2311 If lSeek .AND. (SL1->L1_SITUA $ "OK|X0|X1|X2|X3") //X0 a X3 inclusive, pois nada interfere em cancelamento NFC-e ou SAT
2312     lRet := .T.
2313 EndIf
2314
2315 aAdd(aRet, lRet)
2316 RestArea(aAreaSL1)
2317
2318 Return aRet
2319
2320 //-----
2321 /*{Protheus.doc} STDGetSrNF
2322 Retorna o valor de algum campo da SLG
2323
2324 @Param      cCodEst ->Codigo da estacao
2325 @Param      cCampo -> Campo da SLG que sera retornado
2326 @return     cRet -> Retorno da funcao
2327 @obs
2328 @sample
2329 */
2330 //-----
2331 Static Function STDGetSrNF(cCodEst,cCampo)
2332

```

```

2333 Local cRet      := "" //Variavel de retorno
2334 Local aAreaSLG  := SLG->(GetArea()) //Guarda a area da SLG
2335 Local nPos      := SLG->(FieldPos(cCampo)) //Verifica se o campo existe
2336
2337 Default cCodEst := ""
2338
2339 DbSelectArea("SLG")
2340 DbSetOrder(1)//LG_FILIAL+LG_CODIGO
2341 If SLG->(DbSeek(xFilial("SLG")+cCodEst)) .AND. nPos > 0
2342     cRet := SLG->(FieldGet(nPos))
2343 EndIf
2344
2345 RestArea(aAreaSLG)
2346
2347 Return cRet
2348
2349 /*/{Protheus.doc} STDAltSLX
2350 Rotina para gravar o numero do L1_DOC da Ret no campo LX_CUPOM para que quando integrar a SLX na Ret,
2351 o campo seja compativel com outras tabelas como SFT e SF3. Essa alteração se tornou necessário pois na venda de RPS
2352 o Totvs PDV não pode gravar o campo L1_DOC, e alem disso no momento de integrar a venda do RPS a RET gera a própria sequencia do L1_DOC
2353
2354 @type Static Function
2355 @param cChaveLx, caractere, chave do indice 1 da tabela SLX
2356 @param cL1DOCRet, caractere, numero do L1_DOC retornado pela RET
2357 */
2358 Static Function STDAltSLX(cChaveLx, cL1DOCRet)
2359
2360 Local aAreaSLX := SLX->(GetArea())
2361 Local aSLX := {}
2362 Local lRet := .F.
2363
2364 DbSelectArea("SLX")
2365 DbSetOrder(1) // LX_FILIAL+LX_PDV+LX_CUPOM+LX_SERIE+LX_ITEM
2366
2367 If DbSeek(cChaveLx)
2368     cL1DOCRet :=PADR(cL1DOCRet,TamSX3("LX_CUPOM")[1])
2369     aSLX := {"LX_CUPOM", cL1DOCRet }
2370     lRet := STFSaveTab( "SLX" , aSLX )
2371 EndIf
2372
2373 RestArea(aAreaSLX)
2374
2375 Return lRet
2376
2377
2378 /*/{Protheus.doc} StdGtEstPdvOn
2379 (Retorna array com LG_CODIGO dos Pdvs Online (LG_PDVON = 1)
2380 para que somente movimentos do PDV online seja atualizados na RET via STWUpData)
2381 @type Function
2382 @return aPdvsOn, Array, Array com Pdvs Online
2383 @example
2384 (examples)

```

```
2385      @see (links_or_references)
2386      /*/
2387 Function StdGtEstPdv0n()
2388 Local aPdvs0n := {}
2389 Local aAreaSLG := SLG->(GetArea())
2390
2391 DbSelectArea("SLG")
2392 DbSetOrder(1) //LG_FILIAL+LG_CODIGO
2393
2394 If DbSeek(xFilial("SLG"))
2395     While SLG->( !EOF() ) .AND. SLG->LG_FILIAL == xFilial("SLG")
2396         If SLG->LG_PDVON == '1'
2397             aadd(aPdvs0n, SLG->LG_CODIGO )
2398         Endif
2399         SLG->(DbSkip())
2400     End
2401 EndIf
2402
2403 RestArea(aAreaSLG)
2404
2405 Return aPdvs0n
```

Framework/UpTables/STWUpData.prw

399 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "TBICONN.CH"
3 #INCLUDE "MSOBJECT.CH"
4 #INCLUDE "PARMTYPE.CH"
5 #INCLUDE "STWUPDATA.CH"
6 #INCLUDE "STPOS.CH"
7
8
9 //=====
10 //          Situacoes do _SITUA
11 //
12 // " " - Base Errada, Registro Ignorado.
13 // "00" - Venda Efetuada com Sucesso
14 // "01" - Abertura do Cupom Nao Impressa
15 // "02" - Impresso a Abertura do Cupom
16 // "03" - Item Nao Impresso
17 // "04" - Impresso o Item
18 // "05" - Solicitado o Cancelamento do Item
19 // "06" - Item Cancelado
20 // "07" - Solicitado o Cancelamento do Cupom
21 // "08" - Cupom Cancelado
22 // "09" - Encerrado SL1 (Nao gerado SL4)
23 // "10" - Encerrado a Venda
24 //      Pode nao ter sido impresso o cupom
25 // "TX" - Foi Enviado ao Server (Pdv)
26 // "ER" - Erro ao envia ao server (Pdv)
27 // "EP" - Erro de processamento da venda (Pdv)
28 // "RE" - Ja foi feita nova tentativa de subir (Pdv)
29 // "RX" - Foi Recebido Pelo Server (Server)
30 // "OK" - Foi Processado no Server (Serevr)
31 //      Enviar um OK ao Client que foi Processado
32 // "CP" - Recebido pela Central de PDV
33 // "C0" - Cancelamento a ser enviado pelo PDV
34 // "CX" - Cancelamento enviado pelo PDV
35 //-----
36 // Status da transmissão por NFce Dll que não vão para retaguarda - interno TotvsPDV Mobile
37 // "D1" - NFce Dll - Contingencia off-line - Envia NFce em contingência e consulta Status da NFce para então enviar para retaguarda
38 // "D0" - NFce Dll - Contingencia on-line - Consulta Status da NFce para então enviar para retaguarda
39 // "I8" - NFce Dll - Inutilização não realizada - Tentar novamente - Se deu certo envia venda inutilizada para retaguarda
40 // "DC" - NFce Dll - Cancelamento on line realizado de Nota não enviada - envia o cancelamento novamente
41 // "DX" - NFce Dll - Cancelamento on line realizado de Nota enviada - consulta status da nfce antes de enviar a retaguarda
42 // "D7" - NFce Dll - Cancelamento offline - não enviado - envia o cancelamento em contingencia e depois consulta o status
43 // "D8" - NFce Dll - Cancelamento on line não realizado de nota enviada - tenta realizar o cancelamento e consulta o status
44 // "D9" - NFce - Cancelamento on line não realizado de nota NÃO enviada - tenta cancelar a nota e consulta o status
45 //=====
46
47 /*
48 Exemplo de configuracao do arquivo ini
49
50 [STWUpData]
51 Main=STWUpData
52 Environment=PDVPAF11

```

```

53 nParms=3
54 Parm1=T1
55 Parm2=D MG 01
56 Parm3=001
57
58 [OnStart]
59 Jobs=STWUpData
60 RefreshRate=30
61
62 */
63
64
65 //Exemplo de user function para testes
66 User Function TesteUp()
67     STWUpData( "99" , "01" , "001")
68 Return
69
70
71 //-----
72 /*/{Protheus.doc} STWUpData
73 Efetua o envio das tabelas de vendas e outras para o server
74
75 @param cEmp      Codigo da empresa
76 @param cFil      Codigo da filial
77 @param cPdv      Codigo do PDV
78
79 @return Nil
80 @obs
81 @sample
82 */
83 //-----
84 Function STWUpData( cEmp , cFil , cPdv,cClearCDX)
85
86 Local lPrepEnv      := .F.           // Verifica se deve preparar o Ambiente
87 Local lConnect      := .F.           // Controle de conexao
88 Local oLJCLocker    := Nil           // Obj de Controle de Carga de dados
89 Local lLJCLocker    := .F.           // Utiliza Componente Controle de Carga de dados
90 Local nTamE5Nat      := ""           // Tamanho do campo
91 Local lCentPDV      := .F.           // Eh Central de PDV
92 Local lComCPDV      := .F.           // Usa comunicacao com a Central de PDV
93 Local lRMS          := .F.           //Integracao com a RMS
94 Local lFirst        := .F.           //Integracao com o First
95 Local cIntegration   := "DEFAULT"    //Tipo da Integracao
96 Local nSendOn       := 0             //Retorno como sera a integracao da venda - 0 - via job - 1 online - 2 startjob
97 Local cMvNatTrc     := ""           //Natureza do Troco
98 Local cMvNatSang     := ""           //Natureza da Sangria
99
100 Local cMyUID        := "STWCclearCDX" //ID para reserva no APPSERVER
101 Local cChave        := "ClearCDX"    //CHAVE para reserva no APPSERVER
102 Local nControle     := 0             //Controla para limpeza de CDX
103 Local lRet          := .F.
104 Local nX            := 0

```

```

105
106 Default cEmp      := ""           // Empresa para processamento
107 Default cFil      := ""           // Filial para processamento
108 Default cPdv      := "001"        // Conteudo do terceiro parametro (Parm3 do mp8srv.ini)
109 Default cClearCDX := "0"          //Controla se a limpeza de CDX esta habilitada
110
111
112 //Aguarda para evitar erro de __CInternet
113 Sleep(5000)
114
115 lPrepEnv := !Empty(cEmp) .AND. !Empty(cFil)
116
117
118 If lPrepEnv
119     RPCSetType(3)
120     // "FRT" > Liberacao de acesso PDV cTree para o modulo FrontLoja
121     RpcSetEnv(cEmp,cFil,Nil,Nil,"FRT")
122 ElseIf Empty(cEmpAnt) .AND. Empty(cFilAnt)
123     Conout(STR0001) //"Não foram informados os parametros do processo no arquivo INI"
124 EndIf
125
126 cEstacao := cPdv
127
128 If cClearCDX == "1"
129     Conout("Limpeza dos arquivos CDX na pasta \Data\ ativada")
130     LjGrvLog("STWClearCDX","Limpeza dos arquivos CDX da pasta \Data\ ativada")
131
132     //Cria uma sessão de memoria no appserver
133     lRet := VarSetUID(cMyUID, .T.)
134     If(!lRet)
135         LjGrvLog("STWClearCDX","Erro na criação da sessão: " + cMyUID)
136     EndIf
137
138     //Recupera o valor da chave de memoria
139     lRet := VarGetX(cMyUID, cChave, @nControle)
140
141     If(!lRet)
142         LjGrvLog("STWClearCDX","Erro na recuperação da chave ou chave inexistente")
143     EndIf
144
145     If nControle <> 1
146         Conout("Iniciando limpeza arquivos CDX na pasta \Data\")
147         LjGrvLog("STWClearCDX","Iniciando limpeza arquivos CDX na pasta \Data\")
148
149         STWClearCDX()
150
151         Conout("Finalizando limpeza arquivos CDX na pasta \Data\")
152         LjGrvLog("STWClearCDX","Finalizando limpeza arquivos CDX na pasta \Data\")
153     EndIf
154 EndIf
155
156 lRMS := SuperGetMv("MV_LJ RMS",,.F.)

```

```

157
158 If ExistFunc("STFCfgIntegration")
159     cIntegration := STFCfgIntegration()
160     lRMS := cIntegration == "RMS"
161 Else
162     If SuperGetMv("MV_LJ RMS",,.F.)
163         cIntegration := "RMS"
164         lRMS := .T.
165     EndIf
166
167 Endif
168
169 lFirst := cIntegration == "FIRST"
170 If lPrepEnv
171     cMvNatTrc := LjMExeParam("MV_NATTRC",,"TROCO") //Natureza do Troco
172     cMvNatSang := LjMExeParam("MV_NATSANG",,"SANGRIA") //Natureza da Sangria
173 EndIf
174
175 If !lFirst
176     nTamE5Nat := TamSX3("E5_NATUREZ")[1]
177
178     llJCLocker := ExistFunc("LOJA0051") .And. SuperGetMV( "MV_LJILJL0",,"2" ) == "1"
179     oLJCLocker := If( llJCLocker , LJCGlobalLocker():New(), Nil )
180
181     lCentPDV := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[1] , .F. ) // Eh Central de PDV
182     lComCPDV := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[2] , .F. ) // Usa comunicacao com a central
183
184     // ATENCAO : Cuidado para chamar funcoes atraves da funcao STBRemoteExecute, pois ela pode sobrecarregar o server, somente chamar se necessario
185     ConOut(STR0005+STR0006) //"TOTVS PDV - Estabelecendo conexão com o HOST Superior"
186
187     lConnect := FWHostPing() // Funcao disponibilizada pelo Frame para testar comunicacao (Jun/15)
188
189     If !lConnect //Testa a comunicacao atraves da STFTTestRemoteExecute caso a FWHostPing nao funcione no ambiente do cliente
190         lConnect := STFTTestRemoteExecute(lComCPDV)
191     EndIf
192
193     //Grava no arquivo ini se conseguiu fazer conexao com a Ret = 1 ou se nao tem conexao com a Ret = 0
194     If lConnect
195         WritePProString(CSECA0, CCHAVE, '1', GetAdv97())
196     Else
197         WritePProString(CSECA0, CCHAVE, '0', GetAdv97())
198     EndIf
199
200     If lConnect .AND. (If( llJCLocker, oLJCLocker:GetLock( "LOJA1115ILLock" ), .T. ))
201
202         LjGrvLog("STWUpData", STR0005+STR0002) ////"TOTVS PDV - Conexao estabelecida com o Servidor"
203         ConOut(STR0005+STR0002) //"TOTVS PDV - Conexao estabelecida com o Servidor"
204
205         If lRMS .AND. !lCentPDV
206             //Grava SLX
207             STDRecTabServer("SLX", 3 , "SLX->LX_FILIAL + SLX->LX_SITUA", xFilial("SLX") + "00","TX", "RX", cPDV)
208         EndIf

```

```

209
210 //Subir Vendas
211 STDSalesForUp(cPDV)
212
213 //Grava Cancelamento
214 If !lRMS
215     STDCancSales( cPDV )
216 EndIf
217
218 // Se nao for Central OU FIRST executa demais funcionalidades
219
220 If !lCentPDV
221     STDRecTabServer("SFI", 2, "SFI->FI_FILIAL + SFI->FI_SITUA", xFilial("SFI") + "00",;
222         "TX", "RX", cPDV )
223
224     // Funcao generica
225     STDRecGenTabServer()
226
227     //Grava SLX
228     STDRecTabServer("SLX", 3, "SLX->LX_FILIAL + SLX->LX_SITUA", xFilial("SLX") + "00",;
229         "TX", "RX", cPDV )
230
231     //Grava SLX - Inutilizacao de NFC-e
232     STDRecTabServer( "SLX", 3, "SLX->LX_FILIAL + SLX->LX_SITUA", xFilial("SLX") + "0K" ,;
233         "TX", "OK", cPDV )
234
235     //Grava Sangria
236     STDRecTabServer("SE5", 16, "SE5->E5_FILIAL + SE5->E5_SITUA + SE5->E5_NATUREZ", xFilial("SE5") + StrZero(0,TamSx3("E5_SITUA")[1]) + PADR(cMvNatSang, nTamE5Nat)
237     ,;
238         "OK", "OK", cPDV)
239
240     //Grava Suprimento
241     STDRecTabServer("SE5", 16, "SE5->E5_FILIAL + SE5->E5_SITUA + SE5->E5_NATUREZ", xFilial("SE5") + StrZero(0,TamSx3("E5_SITUA")[1]) + PADR(cMvNatTrc, nTamE5Nat),;
242         "OK", "OK", cPDV)
243
244     If AliasInDic("MFL")
245         //Grava consulta de produtos
246         STDRecTabServer("MFL", 2, "MFL->MFL_FILIAL + MFL->MFL_SITUA", xFilial("MFL") + "00",;
247             "TX", "RX", cPDV )
248     EndIf
249
250     //Grava SLW e SLT caso o parametro MV_LJCONFF esteja ativo e for PDV SigaLoja offline
251     If SuperGetMV( "MV_LJCONFF",,.F. )
252
253         STDRecOpenClose("SLW", 4, "SLW->LW_FILIAL + SLW->LW_SITUA", xFilial("SLW") + "00",;
254             "TX", "RX", cPDV )
255
256         STDRecTabServer("SLT", 2, "SLT->LT_FILIAL + SLT->LT_SITUA", xFilial("SLT") + "00",;
257             "TX", "RX", cPDV )
258     EndIf
259     //Grava Tabela de Apoio PAF-ECF

```

```

260     If AliasInDic( "MDZ" )
261         STDRecTabServer("MDZ", 2, "MDZ->MDZ_FILIAL + MDZ->MDZ_SITUA", xFilial("MDZ") + "00", "OK", "OK", cPDV )
262     EndIf
263
264     // Grava Estorno da venda
265     STDRecEstSale()
266 Else
267     //Sobe o cadastro de cliente que esta na central para a retaguarda
268     STDGrvCli()
269
270     If ExistFunc("StdGtEstPdvOn") // Busca na SLG se existe Estação com Pdv Online
271         aEstsPdvOn = StdGtEstPdvOn()
272         For nX := 1 to Len(aEstsPdvOn) // só pode entrar aqui se for central de pdv e tiver pdv online conectado
273             //Grava Sangria
274             STDRecTabServer("SE5", 16, "SE5->E5_FILIAL + SE5->E5_SITUA + SE5->E5_NATUREZ", xFilial("SE5") + StrZero(0,TamSx3("E5_SITUA")[1]) + PADR(cMvNatSang,
nTamE5Nat) ,;
275                 "OK", "OK", aEstsPdvOn[nX],/*cFunc*/,/*lDefault*/, .T.)
276             //Grava Suprimento
277             STDRecTabServer("SE5", 16, "SE5->E5_FILIAL + SE5->E5_SITUA + SE5->E5_NATUREZ", xFilial("SE5") + StrZero(0,TamSx3("E5_SITUA")[1]) + PADR(cMvNatTrc,
nTamE5Nat),;
278                 "OK", "OK", aEstsPdvOn[nX],/*cFunc*/,/*lDefault*/, .T.)
279             //Grava SLW e SLT caso o parametro MV_LJCONFF esteja ativo e for PDV SigaLoja offline
280             If SuperGetMV( "MV_LJCONFF",,.F. )
281                 STDRecOpenClose("SLW", 4, "SLW->LW_FILIAL + SLW->LW_SITUA", xFilial("SLW") + "00",;
282                     "TX", "RX", aEstsPdvOn[nX], .T. )
283                 STDRecTabServer("SLT", 2, "SLT->LT_FILIAL + SLT->LT_SITUA", xFilial("SLT") + "00",;
284                     "TX", "RX", aEstsPdvOn[nX],/*cFunc*/,/*lDefault*/, .T.)
285             EndIf
286         Next
287     Endif
288
289 EndIf
290
291 If( llJCLocker, oLJCLocker:ReleaseLock( "L0JA1115ILLock" ),) // Componente de Carga de dados
292
293 ConOut(STR0005+STR0003) // TOTVS PDV - Conexao finalizada com o Servidor
294 LjGrvLog("STWUpData", STR0005+STR0003) // TOTVS PDV - Conexao finalizada com o Servidor
295
296 lConnect := .F.
297 Else
298     Conout(STR0004) //"Não foi possivel estabelecer comunicação. Verifique as configurações."
299     Sleep(5000)
300     LjGrvLog("STWUpData", STR0004) //"Não foi possivel estabelecer comunicação. Verifique as configurações."
301 EndIf
302
303 Else
304     cEstacao := cPdv
305     STWUpFirst(cPDV)
306 EndIf
307
308 FWCloseFunctionality() // Desconecta do Host Superior para liberar memoria alocada
309

```

```

310 // Tratamento para evitar problemas de performance,
311 // tinha casos que o RefreshRate estava muito pequeno (ex: 1 )
312 nSendOn := SuperGetMV("MV_LJSEND0",,0) //Retorno como sera a integracao da venda - 0 - via job - 1 online - 2 startjob
313
314 nRefrRate := Val(GetPvProfString("ONSTART","RefreshRate","",GetAdv97()))
315
316 If nSendOn == 0
317     If nRefrRate > 0 .AND. nRefrRate < 50
318         WritePProString("ONSTART","REFRESHRATE","50",GetAdv97())
319     EndIf
320 Else //Quando venda online esta habilitada o JOB se torna contingencia e por este motivo aumenta o tempo do refreshrate
321     If nRefrRate > 0 .AND. nRefrRate < 300
322         WritePProString("ONSTART","REFRESHRATE","100",GetAdv97())
323     EndIf
324 EndIf
325
326 If lPrepEnv
327     RESET ENVIRONMENT // Esta dando erro ao compilar
328 EndIf
329
330 Return Nil
331
332 //-----
333 /*{Protheus.doc} STWUpFirst
334 Workflow de envio da integração First
335
336 @param cPdv          Codigo do PDV
337
338 @return Nil
339 @obs
340 @sample
341 */
342 //-----
343
344 Function STWUpFirst(cPDV)
345
346 // Retirado conteudo da rotina por não existir a função chamada WSINTEGRACAOPDV()
347 // PDV FIRST não está mais ativo
348
349 Return
350
351 //-----
352 /*{Protheus.doc} STWCleanCDX
353 Limpar arquivos CDX da pasta |\data\
354
355 @return Nil
356 @obs
357 @sample
358 */
359 //-----
360
361 Static Function STWCleanCDX()

```

```

362
363 Local aFiles      := {}           //Armazena os arquivos encontrados
364 Local nX          := 0           //variavel para controle
365 Local cDirComp    := "\Data\"    //Variavel que armazena o caminho do diretorio
366 Local cExtArq     := ".cdx"      //Extensão do arquivo que será deletada
367 Local nCount      := 0
368
369 Local cMyUID       := "STWClearCDX" //ID para reserva no APPSERVER
370 Local cChave      := "ClearCDX"    //CHAVE para reserva no APPSERVER
371 Local nValor      := 0             //Valor enviado para a Chave de memoria reservada
372 Local lRet        := .F.
373
374
375 //Varre o diretorio
376 aFiles := Directory( cDirComp + ".*" + cExtArq, "D")
377
378 nCount := Len( aFiles )
379 For nX := 1 to nCount
380
381     LjGrvLog("STWClearCDX","Arquivo: " + aFiles[nX,1] + " - Size: " + AllTrim(Str(aFiles[nX,2])) )
382
383     //Deleta arquivo
384     If FERASE(cDirComp + aFiles[nX,1]) == -1
385         LjGrvLog("STWClearCDX","Falha ao excluir o arquivo: "+ aFiles[nX,1])
386     Else
387         LjGrvLog("STWClearCDX","Arquivo:" + aFiles[nX,1] + " deletado com sucesso.")
388     Endif
389
390 Next nX
391
392 //Atualiza o valor da chave de memoria para evitar que a limpeza das tabelas ocorra mais de uma vez
393 nValor := 1
394 lRet := VarSetX(cMyUID, cChave, nValor)
395 If(!lRet)
396     LjGrvLog("STWClearCDX","Erro na atualização da chave: " + cChave)
397 EndIf
398
399 Return

```